

Machine Programming

Lecture 1 – Overview and Introduction to Synthesis

Ziyang Li

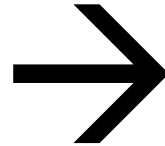
Instructor

Ziyang Li

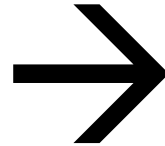
- Assistant professor @JHU CS, 2025-
- Before that: PhD at University of Pennsylvania
- Research areas: Programming Language + Machine Learning
- Favorite PL: Rust & JavaScript



Programming...



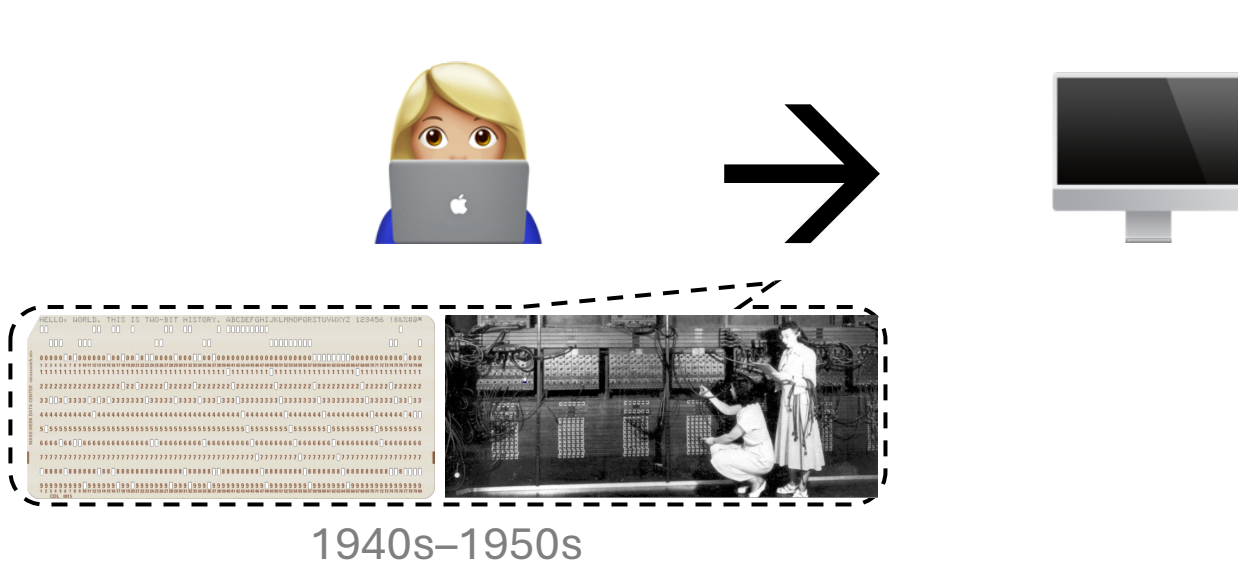
Programming...

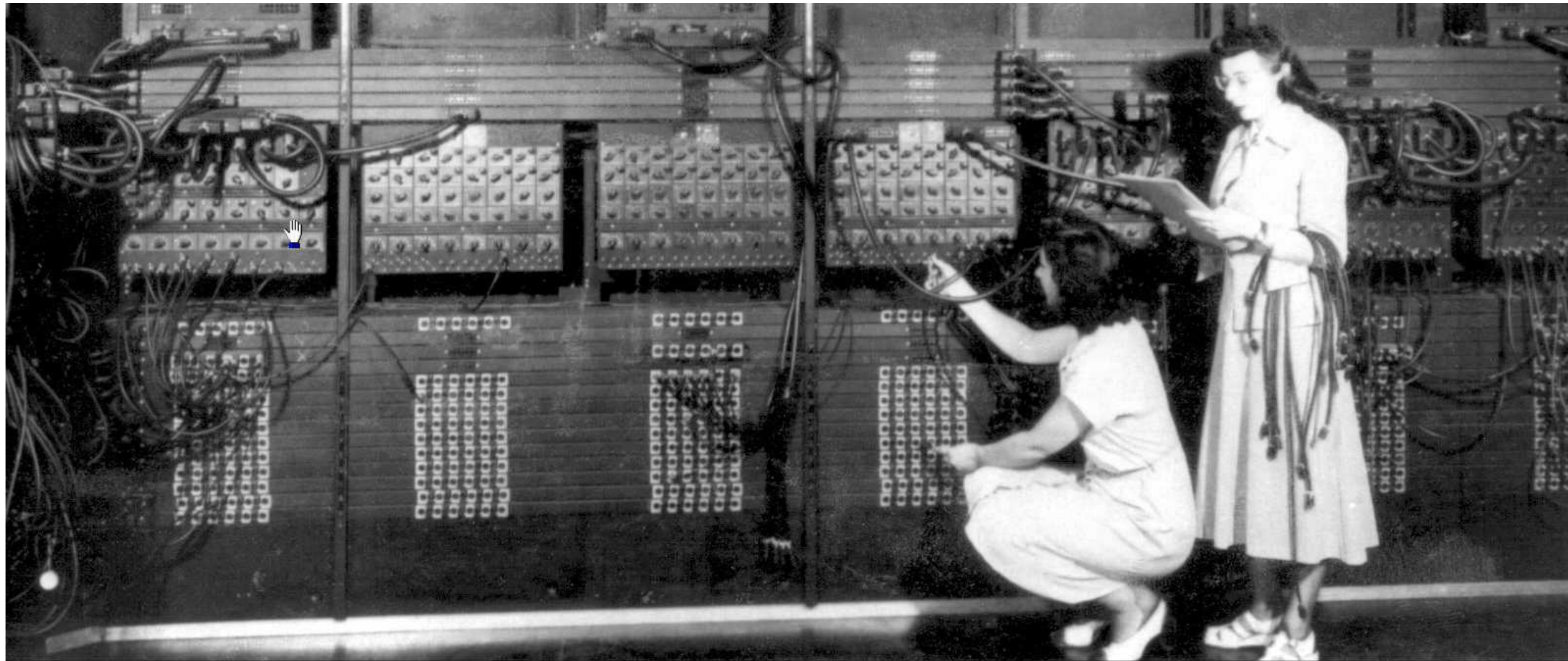


0101010101

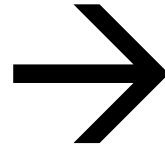
1940s–1950s: Binary

Programming...





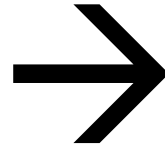
Programming...



0101010101

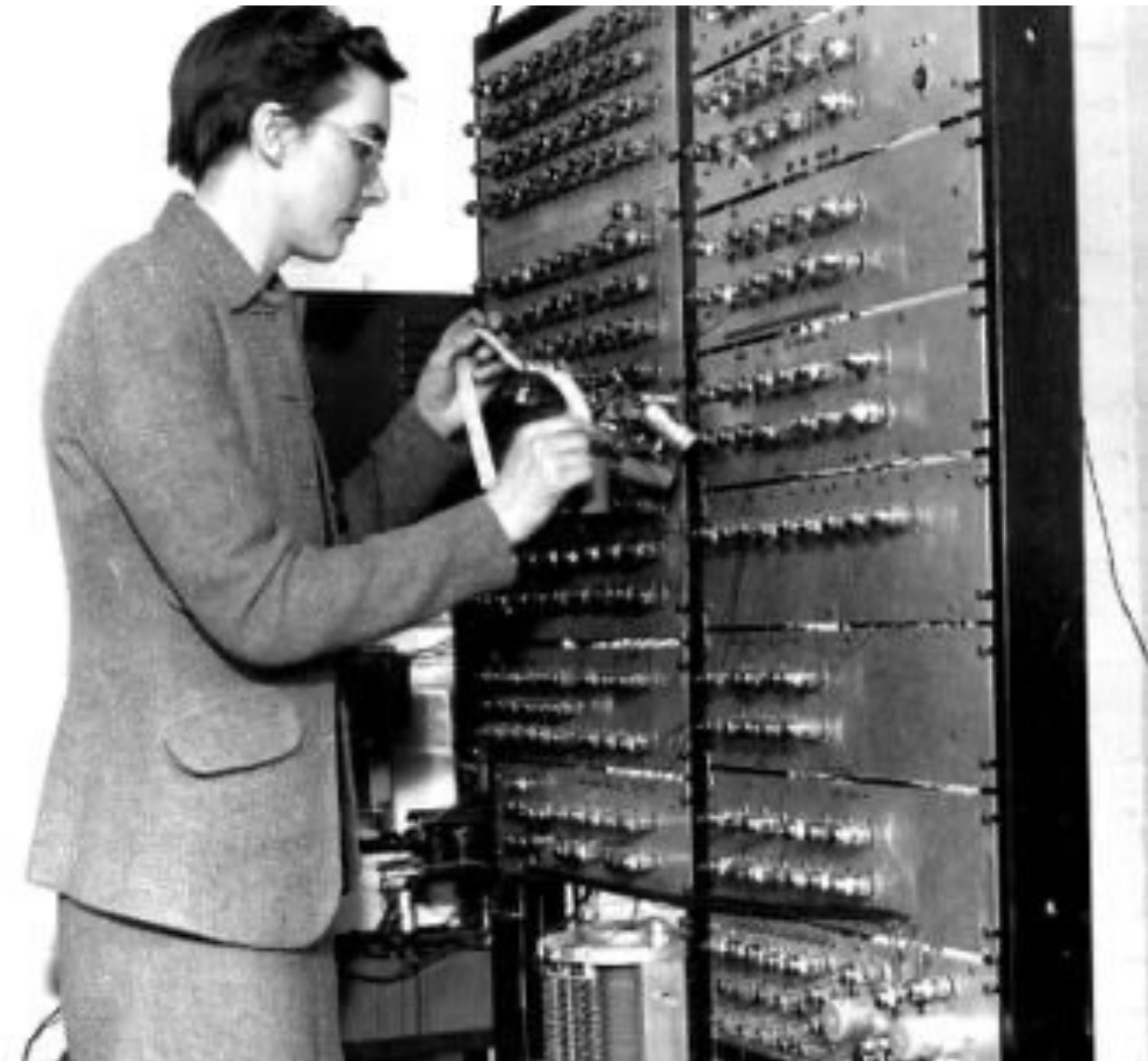
1940s–1950s: Binary

Programming...



ADD R1, R2

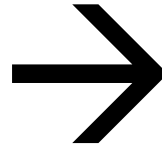
1950s–1960s: Assembly



Kathleen Booth

1922–2022

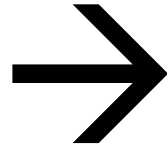
Programming...



ADD R1, R2

1950s–1960s: Assembly

Programming...



Assembler



ADD R1, R2

1950s–1960s: Assembly

0101010101

The FORTRAN Automatic Coding System

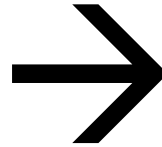
J. W. BACKUS†, R. J. BEEBER†, S. BEST‡, R. GOLDBERG†, L. M. HAIBT†,
H. L. HERRICK†, R. A. NELSON†, D. SAYRE†, P. B. SHERIDAN†,
H. STERN†, I. ZILLER†, R. A. HUGHES§, AND R. NUTT||

INTRODUCTION

THE FORTRAN project was begun in the summer of 1954. Its purpose was to reduce by a large factor the task of preparing scientific problems for IBM's next large computer, the 704. If it were possible for the 704 to code problems for itself and produce as

system is now complete. It has two components: the FORTRAN language, in which programs are written, and the translator or executive routine for the 704 which effects the translation of FORTRAN language programs into 704 programs. Descriptions of the FORTRAN language and the translator form the principal

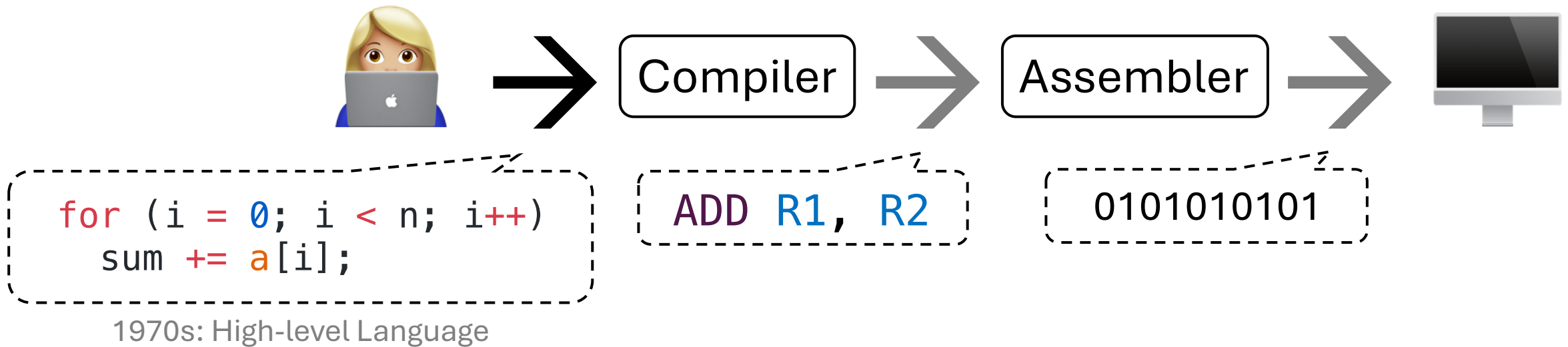
Programming...



```
for (i = 0; i < n; i++)  
    sum += a[i];
```

1970s: High-level Language

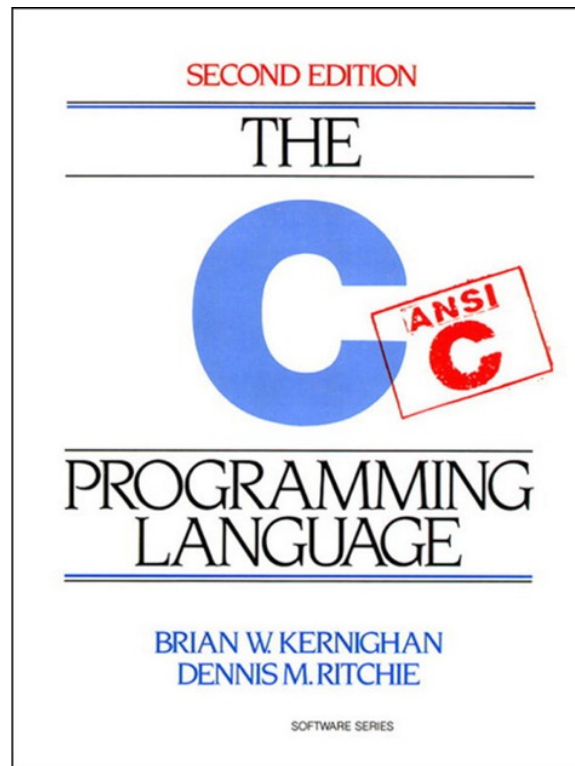
Programming...



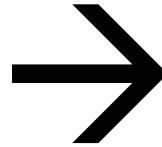


Dennis M. Ritchie

1941–2011



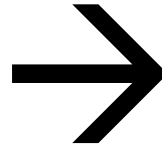
Programming...



```
for (i = 0; i < n; i++)  
    sum += a[i];
```

1970s: High-level Language

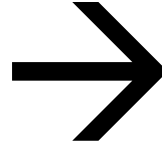
Programming...



```
for i in range(n):  
    sum += a[i]
```

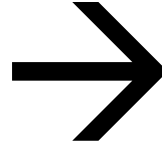
1980s onwards: Modern High-Level Languages

Programming...



```
for i in range(n):  
    sum += a[i]
```

Programming...

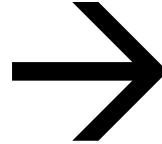


```
for i in range(n):  
    sum += a[i]
```



```
for (var e of a) {  
    sum += e; }  
}
```

Programming...



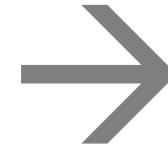
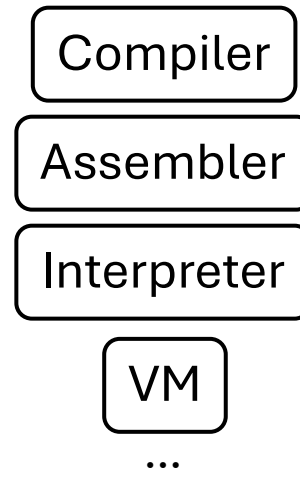
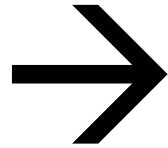
```
for i in range(n):  
    sum += a[i]
```



```
for (var e of a) {  
    sum += e; }  
}
```



```
total :: Num a => [a] -> a  
total a = foldl (+) 0 a
```



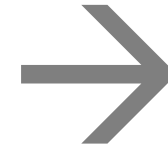
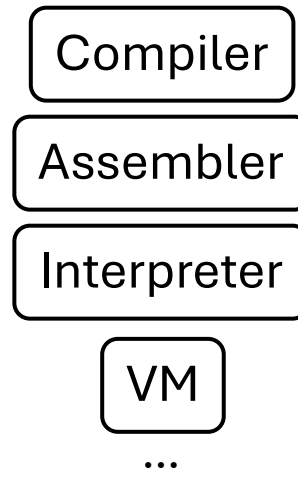
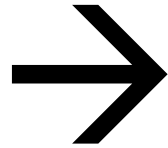
```
for i in range(n):  
    sum += a[i]
```



```
for (var e of a) {  
    sum += e; }  
}
```



```
total :: Num a => [a] -> a  
total a = foldl (+) 0 a
```



```
for i in range(n):  
    sum += a[i]
```



```
for (var e of a) {  
    sum += e; }  
}
```

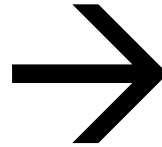


```
total :: Num a => [a] -> a  
total a = foldl (+) 0 a
```

0101010101

ADD R1, R2

Programming...



```
for i in range(n):  
    sum += a[i]
```

Assembly

```
append:
    push ebp
    mov ebp, esp
    push eax
    push ebx
    push len
    call malloc
    mov ebx, [ebp + 12]
    mov [eax + info], ebx
    mov dword [eax + next], 0
    mov ebx, [ebp + 8]
    cmp dword [ebx], 0
    je null_pointer
    mov ebx, [ebx]
```

```
next_element:
    cmp dword [ebx + next], 0
    je found_last
    mov ebx, [ebx + next]
    jmp next_element
```

```
found_last:
    push eax
    push addMes
    call puts
    add esp, 4
    pop eax
    mov [ebx + next], eax
```

```
go_out:
    pop ebx
    pop eax
    mov esp, ebp
    pop ebp
    ret 8
```

```
null_pointer:
    push eax
    push nullMes
    call puts
    add esp, 4
    pop eax
    mov [ebx], eax
    jmp go_out
```

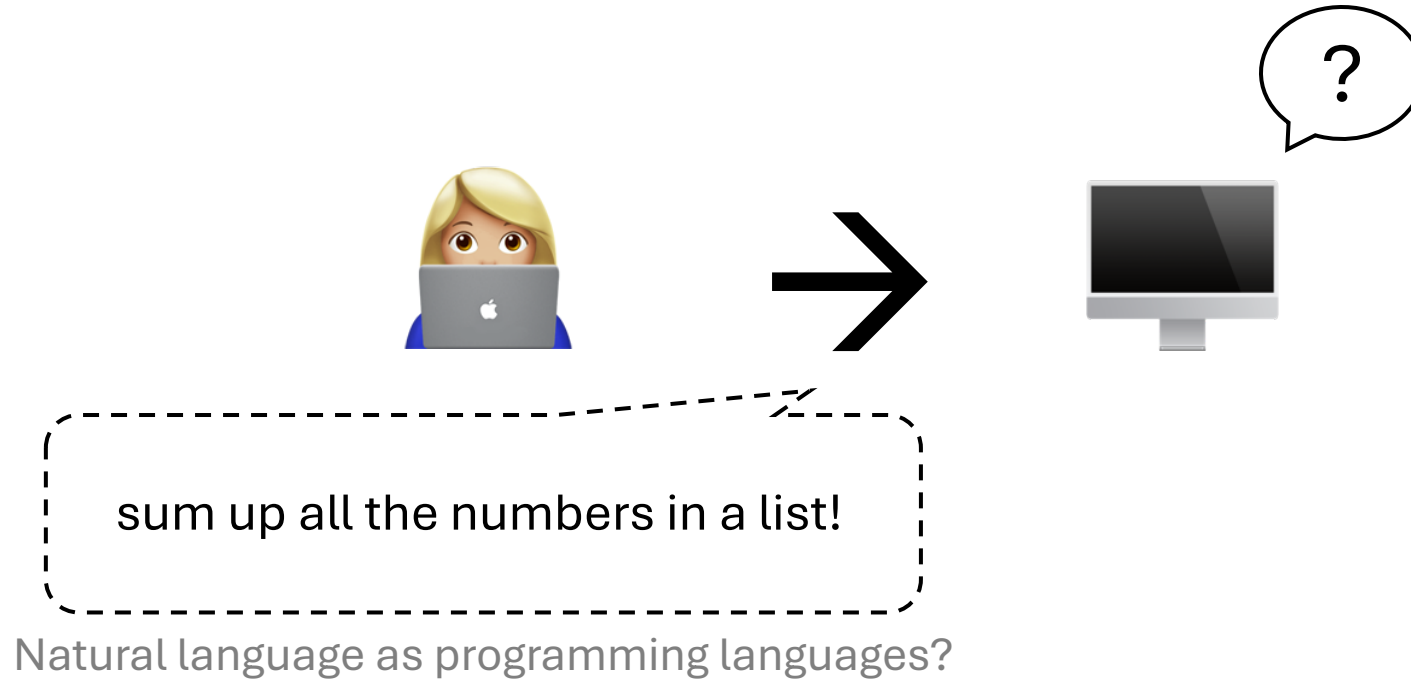
C

```
void insert(node *xs, int x) {
    node *new;
    node *temp;
    node *prev;
    new = (node *)malloc(sizeof(node));
    if(new == NULL) {
        printf("Insufficient memory.");
        return;
    }
    new->val = x;
    new->next = NULL;
    if (xs == NULL) {
        xs = new;
    } else if(x < xs->val) {
        new->next = xs;
        xs = new;
    } else {
        prev = xs;
        temp = xs->next;
        while(temp != NULL && x > temp->val) {
            prev = temp;
            temp = temp->next;
        }
        if(temp == NULL) {
            prev->next = new;
        } else {
            new->next = temp;
            prev->next = new;
        }
    }
}
```

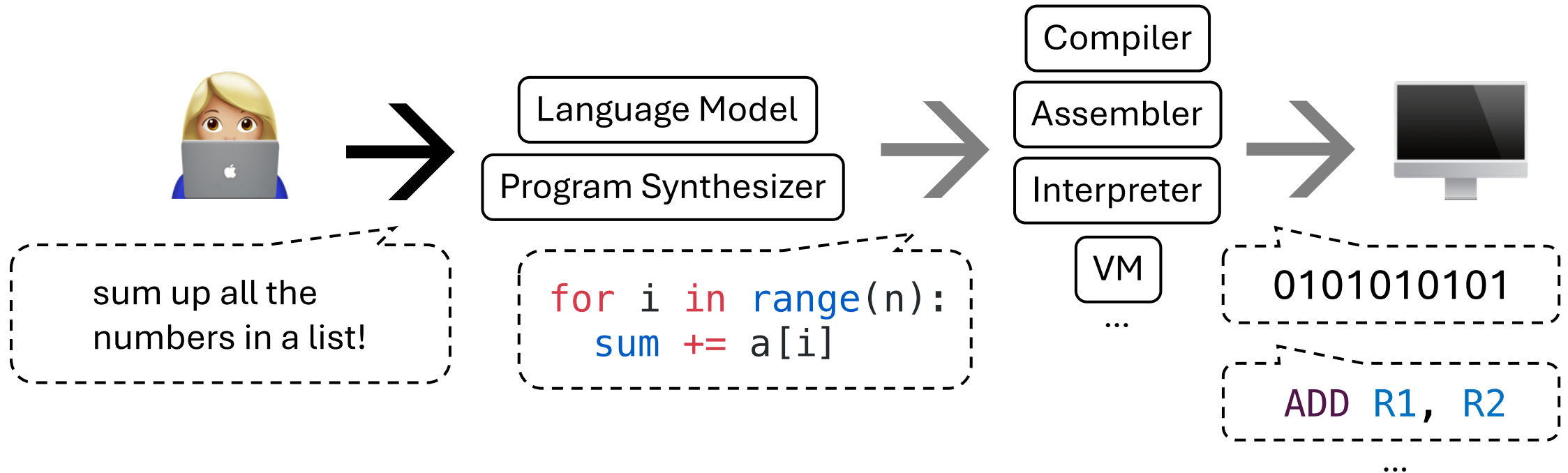
Haskell

```
insert x [] = [x]
insert x (y:ys)
    | x ≤ y = x:y:ys
    | otherwise = y:(insert x ys)
```

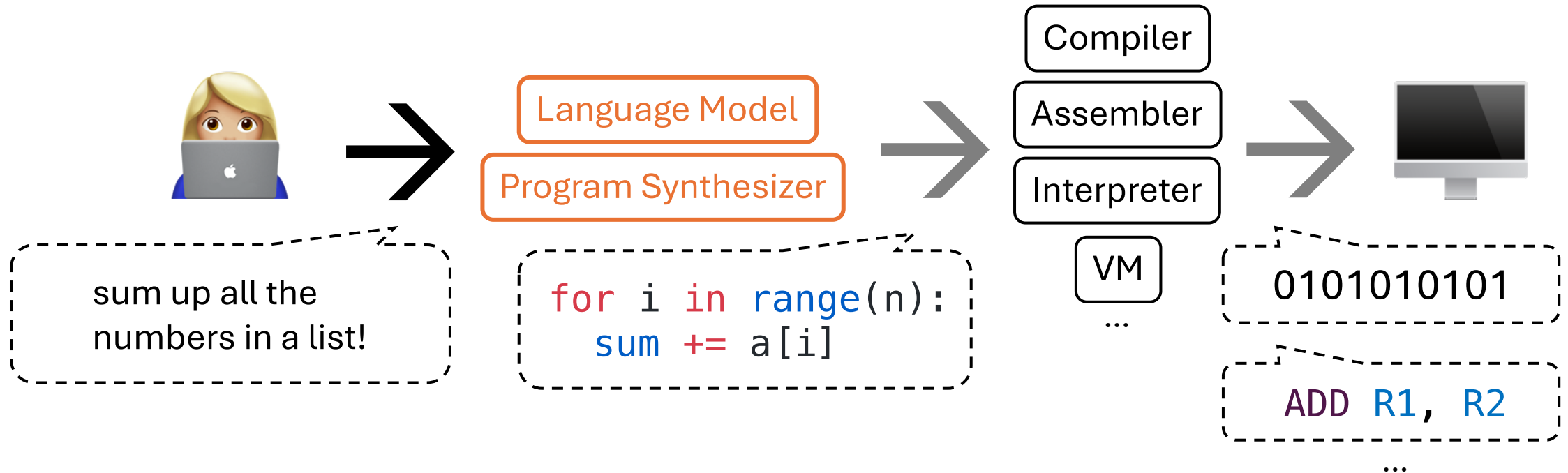
Programming → Talking to Computers

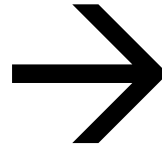


Programming → Talking to Computers

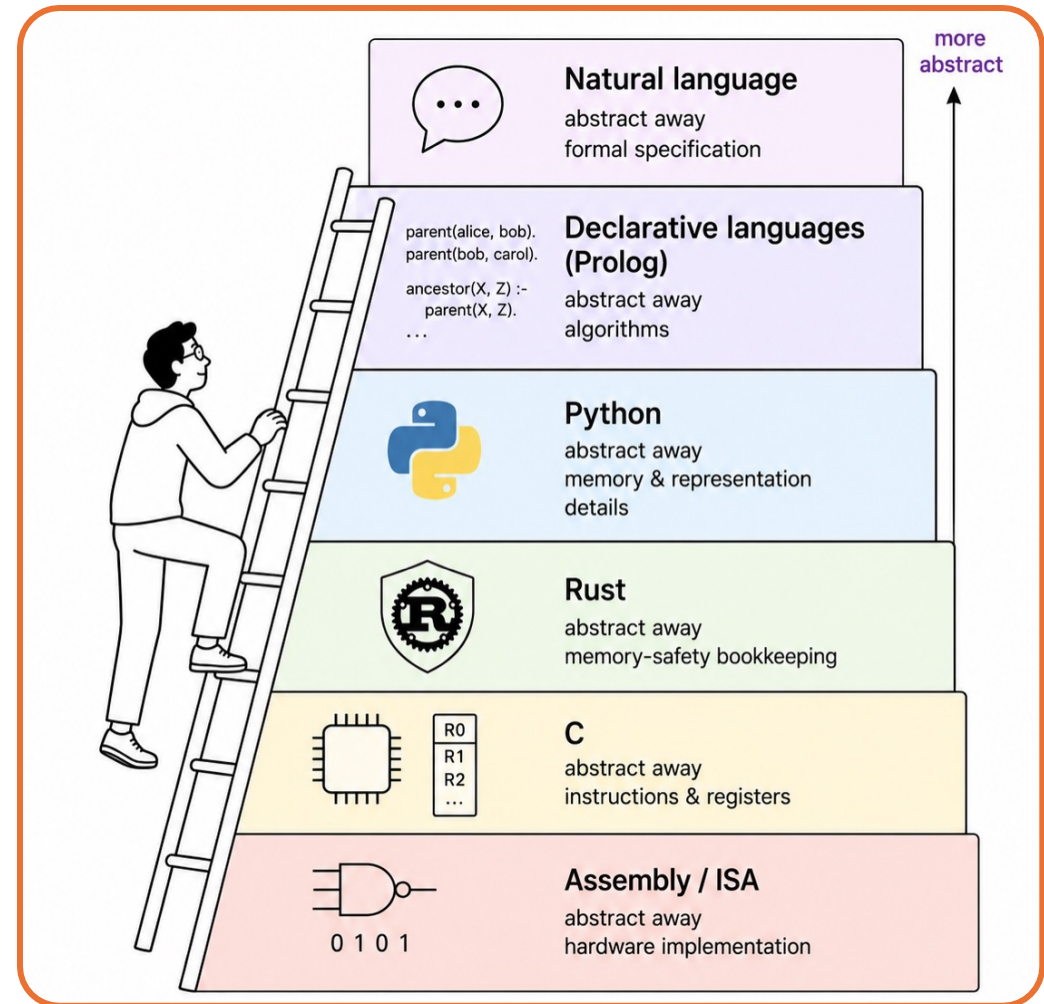
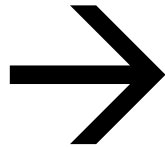


Programming → Talking to Computers





Climb the **Abstraction** ladder



Course Roadmap & Logistics

Logistics

- Lecture
 - When: Tue/Thu 12:00 – 1:15pm
 - Where: Latrobe 107
- Office Hours
 - Instructor: Fri 10:00 – 11:00am, Zoom (<https://wse.zoom.us/my/ziyang>)
 - TA: Thu 10:00 – 11:00am, CA: Wed 11:00am – 12:00pm, Zoom
- Course Website
 - <https://machine-programming.github.io>
 - Discussions: [courselore](#)

Roadmap

Foundations + Applications

of machine programming

Topics

- Foundations

- **Programming language, syntax and semantics**
- **Classical synthesis**, e.g., inductive synthesis, bottom-up and top-down synthesis, type-guided synthesis, specification guided synthesis
- **LLM-based synthesis**, e.g., natural language guided synthesis, agentic frameworks, model context protocol, language server protocol

- Applications

- **Software Engineering & Security**, e.g., testing, transpilation, verification
- **Math & Science**, e.g., theorem-proving, simulation, high-performance computing
- **Data Wrangling**, e.g., database querying, code querying
- **Planning & Cyber-Physical Systems**, e.g. robotics, simulation, reward

Topics that YOU Can Explore

- Search algorithms
- Neurosymbolic method
- Feedback engineering
- Example guided synthesis
- Probabilistic method
- Human-in-the-loop synthesis / Vibe Coding

1. Synthesis methodologies

3. Synthesis applications

- General programming
- Data wrangling
- Program repair & optimization
- Verification & security
- Scientific domains & math
- Embodied AI & robotics
- Creative coding
- Visualization, games, and graphics

- MCP
- Reinforcement learning
- Agentive behavior
- Evaluation of coding capabilities
- Adversarial attack
- Prompting strategies
- Synthetic data generation
- Context engineering
- Fine-tuning strategies

2. Foundation models

4. Programming languages

- Syntax & semantics
- Type systems for synthesis
- Synthesis for domain-specific languages
- Synthesis for low-resource languages
- Language design
- Specification languages
- Language server protocol
- Safety properties

Topics that YOU Can Explore

- Search algorithms
- Feedback engineering
- Example guided synthesis
- Human-in-the-loop synthesis / Vibe Coding
- Neurosymbolic method
- Probabilistic method

1. Synthesis methodologies

3. Synthesis applications

- General programming
- Program repair & optimization
- Verification & security
- Embodied AI & robotics
- Visualization, games, and graphics
- Data wrangling
- Scientific domains & math
- Creative coding

- MCP
- Evaluation of coding capabilities
- Prompting strategies
- Context engineering
- Reinforcement learning
- Adversarial attack
- Synthetic data generation
- Fine-tuning strategies
- Agentic behavior

2. Foundation models

4. Programming languages

- Syntax & semantics
- Synthesis for domain-specific languages
- Synthesis for low-resource languages
- Language design
- Language server protocol
- Type systems for synthesis
- Specification languages
- Safety properties

Topics that YOU Can Explore

- Search algorithms
- Neurosymbolic method
- Feedback engineering
- Example guided synthesis
- Probabilistic method
- Human-in-the-loop synthesis / Vibe Coding

1. Synthesis methodologies

3. Synthesis applications

- General programming
- Data wrangling
- Program repair & optimization
- Verification & security
- Scientific domains & math
- Embodied AI & robotics
- Creative coding
- Visualization, games, and graphics

- MCP
- Reinforcement learning
- Evaluation of coding capabilities
- Prompting strategies
- Context engineering
- Agentive behavior
- Adversarial attack
- Synthetic data generation
- Fine-tuning strategies

2. Foundation models

4. Programming languages

- Syntax & semantics
- Type systems for synthesis
- Synthesis for domain-specific languages
- Synthesis for low-resource languages
- Language design
- Specification languages
- Language server protocol
- Safety properties

Topics that YOU Can Explore

- Search algorithms
- Neurosymbolic method
- Feedback engineering
- Example guided synthesis
- Probabilistic method
- Human-in-the-loop synthesis / Vibe Coding

1. Synthesis methodologies

3. Synthesis applications

- General programming
- Data wrangling
- Program repair & optimization
- Verification & security
- Scientific domains & math
- Embodied AI & robotics
- Creative coding
- Visualization, games, and graphics

- MCP
- Reinforcement learning
- Agentive behavior
- Evaluation of coding capabilities
- Adversarial attack
- Prompting strategies
- Synthetic data generation
- Context engineering
- Fine-tuning strategies

2. Foundation models

4. Programming languages

- Syntax & semantics
- Type systems for synthesis
- Synthesis for domain-specific languages
- Synthesis for low-resource languages
- Language design
- Specification languages
- Language server protocol
- Safety properties

Topics that YOU Can Explore

- Search algorithms
- Neurosymbolic method
- Feedback engineering
- Example guided synthesis
- Probabilistic method
- Human-in-the-loop synthesis / Vibe Coding

1. Synthesis methodologies

- General programming
- Data wrangling
- Program repair & optimization
- Verification & security
- Scientific domains & math
- Embodied AI & robotics
- Creative coding
- Visualization, games, and graphics

3. Synthesis applications

- MCP
- Reinforcement learning
- Agentic behavior
- Evaluation of coding capabilities
- Adversarial attack
- Prompting strategies
- Synthetic data generation
- Context engineering
- Fine-tuning strategies

2. Foundation models

4. Programming languages

- Syntax & semantics
- Type systems for synthesis
- Synthesis for domain-specific languages
- Synthesis for low-resource languages
- Language design
- Specification languages
- Language server protocol
- Safety properties

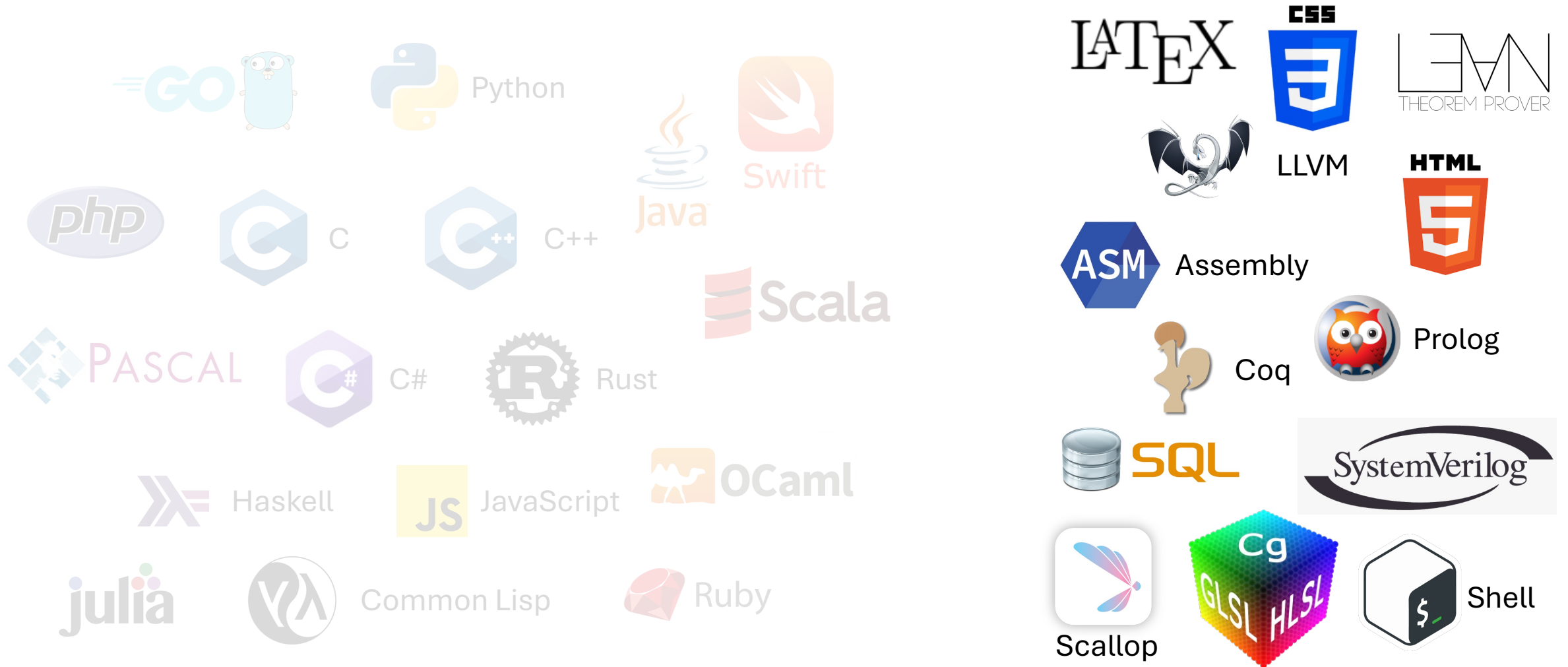
Programming Languages

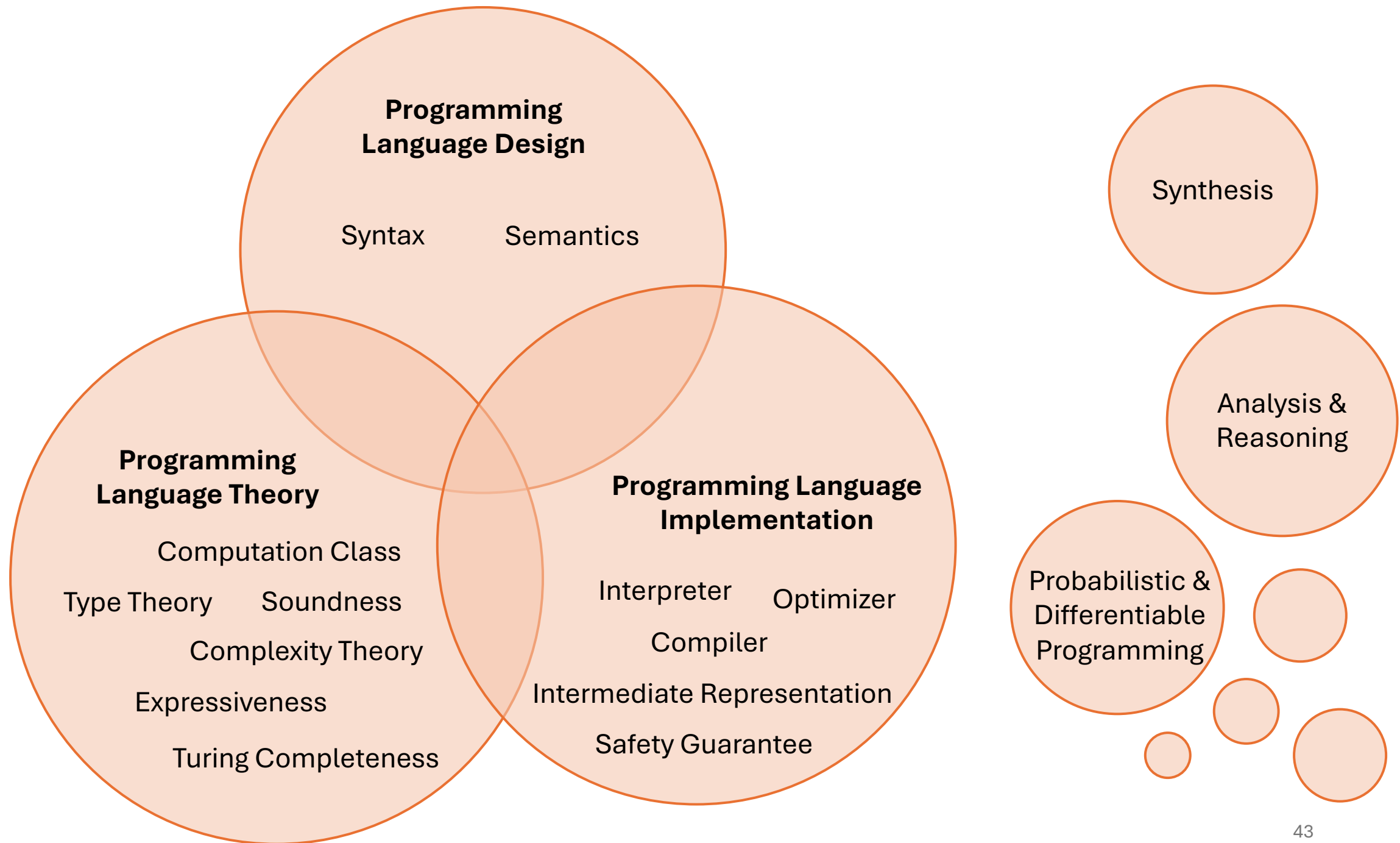


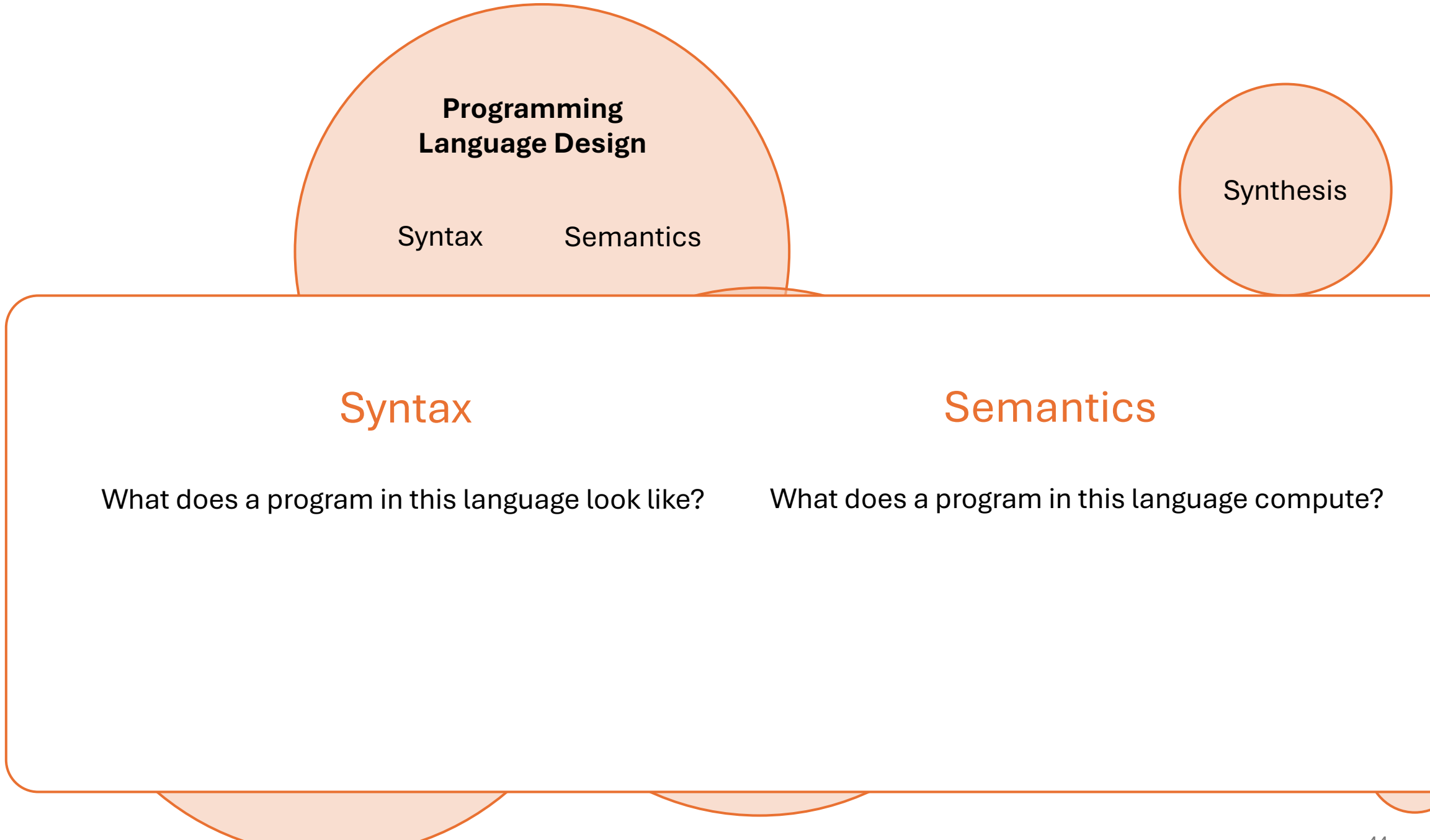
General Purpose Programming Languages



Domain Specific Programming Languages







The diagram illustrates the components of Programming Language Design. At the top, a large light-orange semi-circle contains the text 'Programming Language Design' in bold, with 'Syntax' and 'Semantics' below it. To the right, a smaller light-orange circle contains the text 'Synthesis'. Below these, a large light-orange rounded rectangle contains two columns: 'Syntax' and 'Semantics'. Under 'Syntax' is the question 'What does a program in this language look like?'. Under 'Semantics' is the question 'What does a program in this language compute?'. The entire diagram is framed by light-orange semi-circles at the top and bottom.

Programming Language Design

Syntax

Semantics

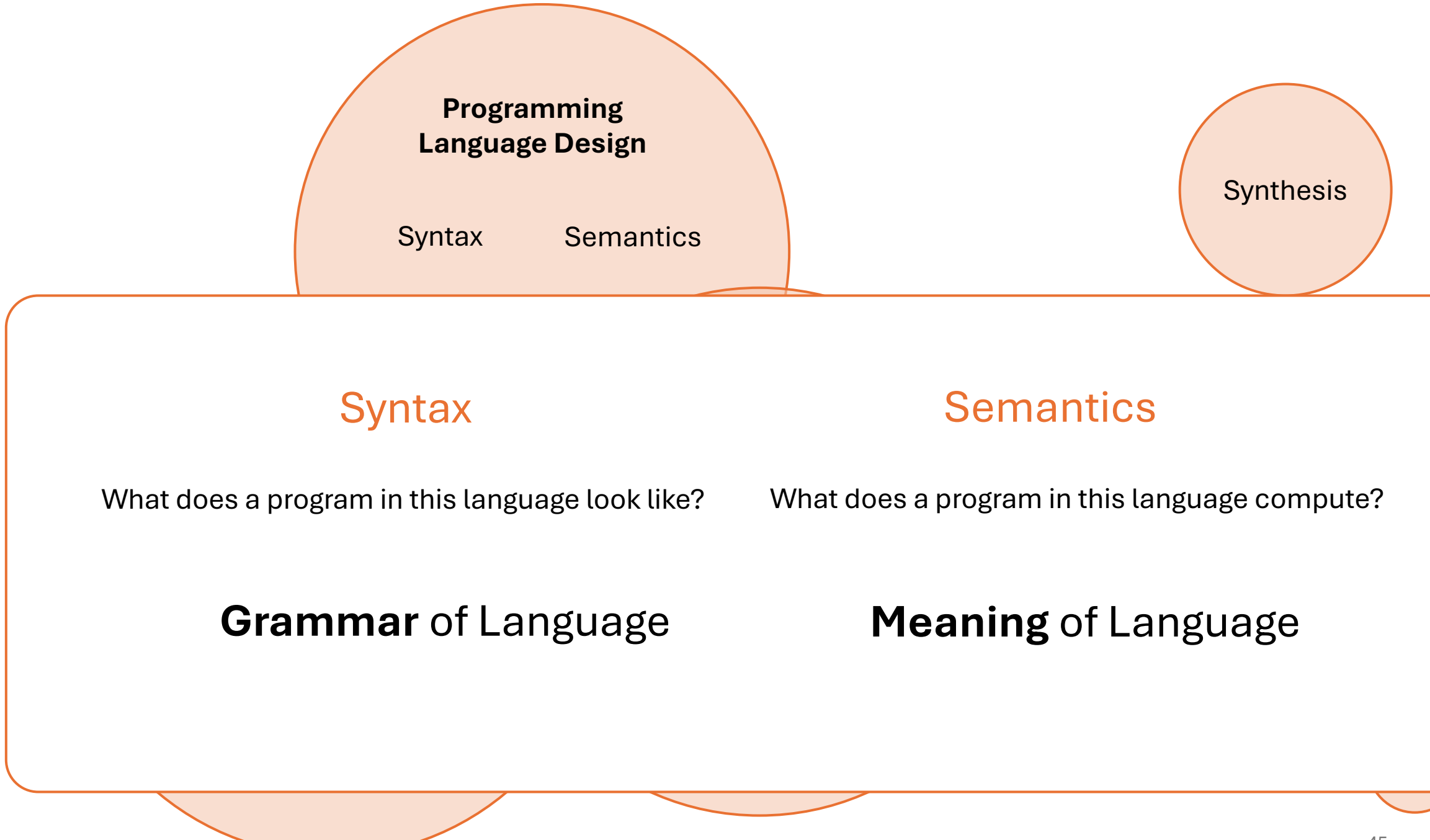
Synthesis

Syntax

What does a program in this language look like?

Semantics

What does a program in this language compute?



The diagram illustrates the components of Programming Language Design. At the top, a large orange semi-circle contains the text 'Programming Language Design'. Below this, the words 'Syntax' and 'Semantics' are positioned. To the right, a separate orange circle contains the word 'Synthesis'. The main body of the diagram is a large white rounded rectangle with an orange border. Inside this rectangle, the words 'Syntax' and 'Semantics' are written in orange at the top. Below 'Syntax' is the question 'What does a program in this language look like?' and below 'Semantics' is 'What does a program in this language compute?'. At the bottom of the rectangle, 'Grammar of Language' is aligned under the syntax question, and 'Meaning of Language' is aligned under the semantics question. The entire diagram is framed by orange semi-circular shapes at the top and bottom.

Programming Language Design

Syntax

Semantics

Synthesis

Syntax

Semantics

What does a program in this language look like?

What does a program in this language compute?

Grammar of Language

Meaning of Language

```

if_stmt:
    | 'if' named_expression ':' block elif_stmt
    | 'if' named_expression ':' block [else_block]
elif_stmt:
    | 'elif' named_expression ':' block elif_stmt
    | 'elif' named_expression ':' block [else_block]
else_block:
    | 'else' ':' block

# While statement
# -----

while_stmt:
    | 'while' named_expression ':' block [else_block]

# For statement
# -----

for_stmt:
    | 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [else_block]
    | 'async' 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [el

# With statement
# -----

with_stmt:
    | 'with' '(' ','.with_item+ ','? ')' ':' [TYPE_COMMENT] block
    | 'with' ','.with_item+ ':' [TYPE_COMMENT] block
    | 'async' 'with' '(' ','.with_item+ ','? ')' ':' block
    | 'async' 'with' ','.with_item+ ':' [TYPE_COMMENT] block

with_item:
    | expression 'as' star_target &(',' | ')') | ':'
    | expression

# Try statement
# -----

try_stmt:
    | 'try' ':' block finally_block
    | 'try' ':' block except_block+ [else_block] [finally_block]
    | 'try' ':' block except_star_block+ [else_block] [finally_block]

```

What language is this?

Concrete and Abstract Syntax

Python Grammar (Concrete Syntax)

```
if_stmt:
    'if' named_expression ':' block elif_stmt
    'if' named_expression ':' block [else_block]
elif_stmt:
    'elif' named_expression ':' block elif_stmt
    'elif' named_expression ':' block [else_block]
else_block:
    'else' ':' block

# While statement
# -----
while_stmt:
    'while' named_expression ':' block [else_block]

# For statement
# -----
for_stmt:
    'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [else_block]
    'async' 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMMENT] block [el

# With statement
# -----
with_stmt:
    'with' '(' ',' with_item+ ',' '?' ')' ':' [TYPE_COMMENT] block
    'with' ',' with_item+ ':' [TYPE_COMMENT] block
    'async' 'with' '(' ',' with_item+ ',' '?' ')' ':' block
    'async' 'with' ',' with_item+ ':' [TYPE_COMMENT] block

with_item:
    expression 'as' star_target &(',' | ')') | ':'

# Try statement
# -----
try_stmt:
    'try' ':' block finally_block
    'try' ':' block except_block+ [else_block] [finally_block]
    'try' ':' block except_star_block+ [else_block] [finally_block]
```

<https://docs.python.org/3/reference/grammar.html>

Python Abstract Syntax

```
-- use 'orelse' because else is a keyword in target languages
For(expr target, expr iter, stmt* body, stmt* orelse, string?
AsyncFor(expr target, expr iter, stmt* body, stmt* orelse, st
While(expr test, stmt* body, stmt* orelse)
If(expr test, stmt* body, stmt* orelse)
With(withitem* items, stmt* body, string? type_comment)
AsyncWith(withitem* items, stmt* body, string? type_comment)

Match(expr subject, match_case* cases)

Raise(expr? exc, expr? cause)
Try(stmt* body, excepthandler* handlers, stmt* orelse, stmt*
TryStar(stmt* body, excepthandler* handlers, stmt* orelse, st
Assert(expr test, expr? msg)

Import(alias* names)
ImportFrom(identifier? module, alias* names, int? level)

Global(identifier* names)
Nonlocal(identifier* names)
Expr(expr value)
Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string the parser
attributes (int lineno, int col_offset, int? end_lineno, int? e

-- BoolOp() can use left & right?
expr = BoolOp(boolop op, expr* values)
NamedExpr(expr target, expr value)
BinOp(expr left, operator op, expr right)
UnaryOp(unaryop op, expr operand)
Lambda(arguments args, expr body)
IfExp(expr test, expr body, expr orelse)
Dict(expr* keys, expr* values)
Set(expr* elts)
ListComp(expr elt, comprehension* generators)
```

<https://docs.python.org/3/library/ast.html>

Formal Semantics

WebAssembly (WASM)

Datafun

Reduction

$$\begin{array}{c}
 \frac{s; v^*; e^* \hookrightarrow_i s'; v'^*; e'^*}{s; v^*; L^k[e^*] \hookrightarrow_i s'; v'^*; L^k[e'^*]} \quad \frac{s; v^*; e^* \hookrightarrow_i s'; v'^*; e'^*}{s; v_0^*; \text{local}_n\{i; v^*\} e^* \text{ end} \hookrightarrow_j s'; v_0^*; \text{local}_n\{i; v'^*\} e'^* \text{ end}} \quad \boxed{s; v^*; e^* \hookrightarrow_i s; v^*; e^*} \\
 \\
 L^0[\text{trap}] \hookrightarrow \text{trap} \quad \text{if } L^0 \neq [-] \\
 \frac{(t.\text{const } c) t.\text{unop}}{(t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop}} \hookrightarrow t.\text{const unop}_t(c) \\
 \frac{(t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop}}{(t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop}} \hookrightarrow t.\text{const } c \quad \text{if } c = \text{binop}_t(c_1, c_2) \\
 \frac{(t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop}}{(t.\text{const } c_1) (t.\text{const } c_2) t.\text{binop}} \hookrightarrow \text{trap} \quad \text{otherwise}
 \end{array}$$

Expression semantics

$$\begin{array}{c}
 \boxed{\alpha : \mathcal{U} \rightarrow \mathcal{U}, \quad \beta : \mathcal{U} \rightarrow \text{Bool}, \quad g : \mathcal{U} \rightarrow \mathcal{U}, \quad \llbracket e \rrbracket : \mathcal{F}_T \rightarrow \mathcal{U}_T} \\
 \\
 \frac{(t.\text{const } c) \quad t :: p(u) \in F_T}{(t.\text{const } c) \quad t :: p(u) \in F_T} \text{ (PREDICATE)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad \beta(u) = \text{true}}{t :: u \in \llbracket e \rrbracket(F_T)} \text{ (SELECT)} \quad \frac{t :: u \in \llbracket e \rrbracket(F_T) \quad u' = \alpha(u)}{t :: u' \in \llbracket \pi_\alpha(e) \rrbracket(F_T)} \text{ (PROJECT)} \\
 \\
 \frac{t :: u_2 \in \llbracket e_2 \rrbracket(F_T)}{e_1 \times e_2(F_T)} \text{ (PRODUCT)} \quad \frac{t_2 :: u \in \llbracket e_2 \rrbracket(F_T)}{e_1 - e_2(F_T)} \text{ (DIFF-2)} \quad \frac{u \in g(\{u_i\}_{i=1}^n)}{u \in g(\{u_i\}_{i=1}^n)} \text{ (AGGREGATE)}
 \end{array}$$

SQL

$$\left[\begin{array}{l} \text{SELECT DISTINCT } \alpha : \beta' \mid * \\ \text{FROM } \tau : \beta \text{ WHERE } \theta \end{array} \right]_{D, \eta, x} = \varepsilon \left(\left[\begin{array}{l} \text{SELECT } \alpha : \beta' \mid * \\ \text{FROM } \tau : \beta \text{ WHERE } \theta \end{array} \right]_{D, \eta, x} \right)$$

Signature

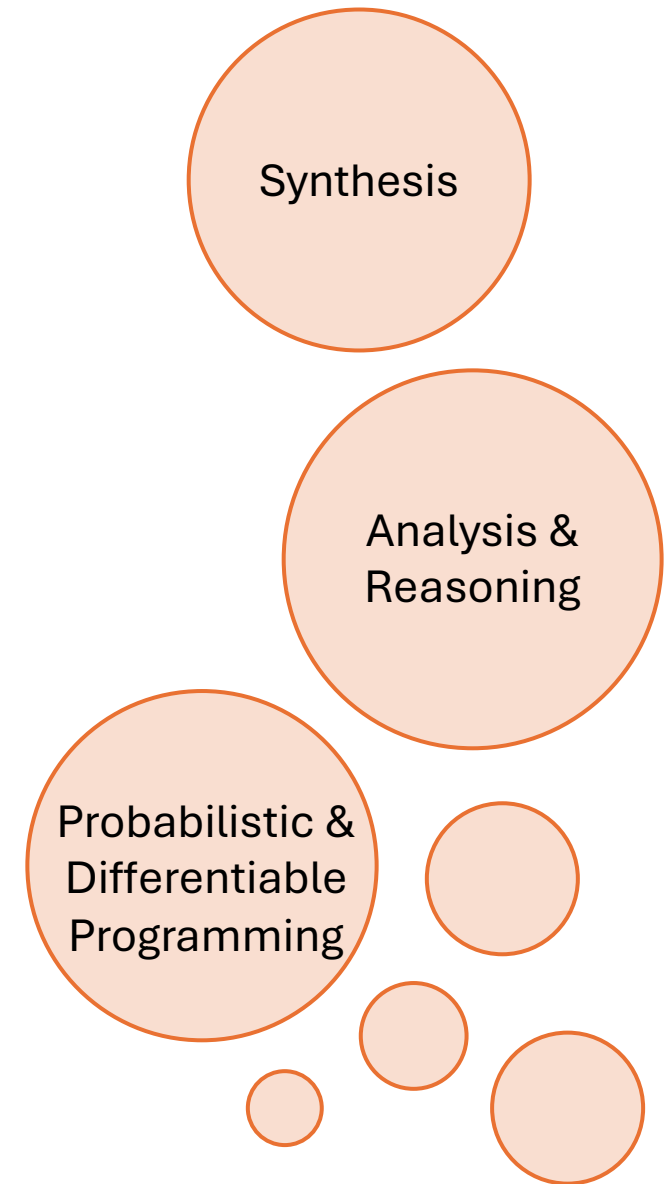
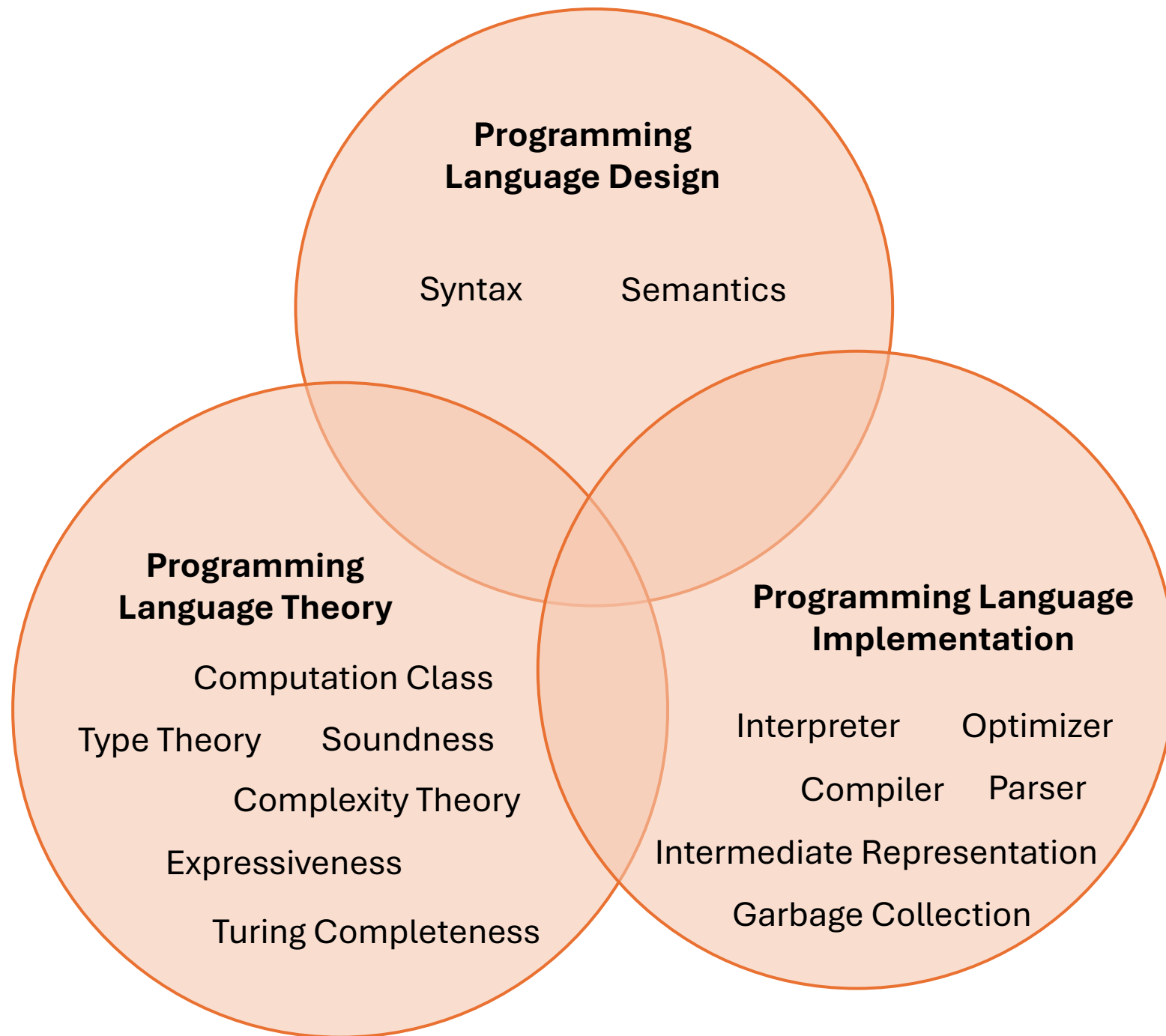
$\text{alloc}(\tau_1, \dots, \tau_n)(\bar{r}_n, S)$
 $\bar{d}_m \leftarrow \text{eval}(\alpha_{n,m})(\bar{s}_n)$
 $\bar{d}_n \leftarrow \text{gather}(i, \bar{s}_n)$
 $d \leftarrow \text{gather}(\alpha_{n,1})(\bar{i}_n, \bar{s}_n)$
 $\text{store}(\rho)(\bar{s}_n, s_r)$
 $[\bar{s}_n, s_r] = \text{load}(\rho)()$
 $\bar{d}_n \leftarrow \text{build}(\bar{s}_n)$
 $\bar{d}_n \leftarrow \text{count}(\bar{b}_n, h, \bar{a}_n)$
 $\bar{d}_n \leftarrow \text{scan}(s)$
 $[d_l, d_r] \leftarrow \text{join}(W)(\bar{b}_m)$
 $\bar{d}_n \leftarrow \text{copy}(\bar{s}_n)$
 $\bar{d}_n \leftarrow \text{sort}(\bar{s}_n)$
 $[\bar{d}_n, s] \leftarrow \text{unique}(\sigma)(\bar{s}_n)$
 $\bar{d}_n \leftarrow \text{merge}(\bar{a}_n, \bar{b}_n)$

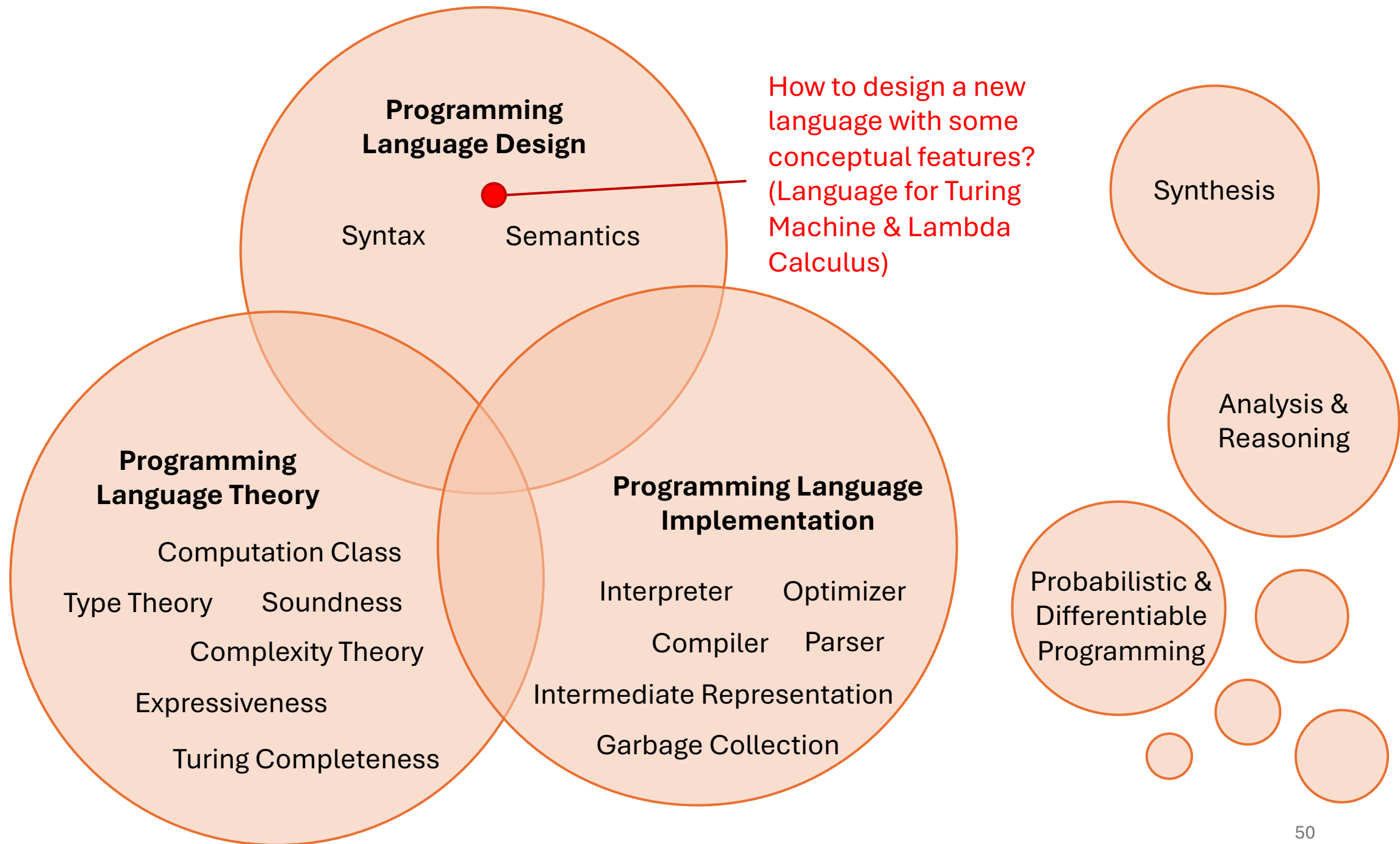
$$\begin{array}{ll}
 \llbracket A \rrbracket & \in \text{Poset}_0 \\
 \llbracket 2 \rrbracket & = 2 \\
 \llbracket N \rrbracket & = N_{\leq} \\
 \llbracket \text{str} \rrbracket & = \text{Disc } S \\
 \llbracket A \times B \rrbracket & = \llbracket A \rrbracket \times \llbracket B \rrbracket \\
 \llbracket A + B \rrbracket & = \llbracket A \rrbracket + \llbracket B \rrbracket \\
 \llbracket A \xrightarrow{+} B \rrbracket & = \llbracket A \rrbracket \Rightarrow \llbracket B \rrbracket \\
 \llbracket A \rightarrow B \rrbracket & = \text{Disc } |\llbracket A \rrbracket| \Rightarrow \llbracket B \rrbracket \\
 \llbracket \{A\} \rrbracket & = \mathcal{P}_{\text{fin}} |\llbracket A \rrbracket| \\
 \\
 \llbracket \Delta \rrbracket, \llbracket \Gamma \rrbracket & \in \text{Poset}_0 \\
 \llbracket \cdot \rrbracket & = 1 \\
 \llbracket \Delta, x : A \rrbracket & = \llbracket \Delta \rrbracket \times \llbracket A \rrbracket \\
 \llbracket \Gamma, x : A \rrbracket & = \llbracket \Gamma \rrbracket \times \llbracket A \rrbracket
 \end{array}$$

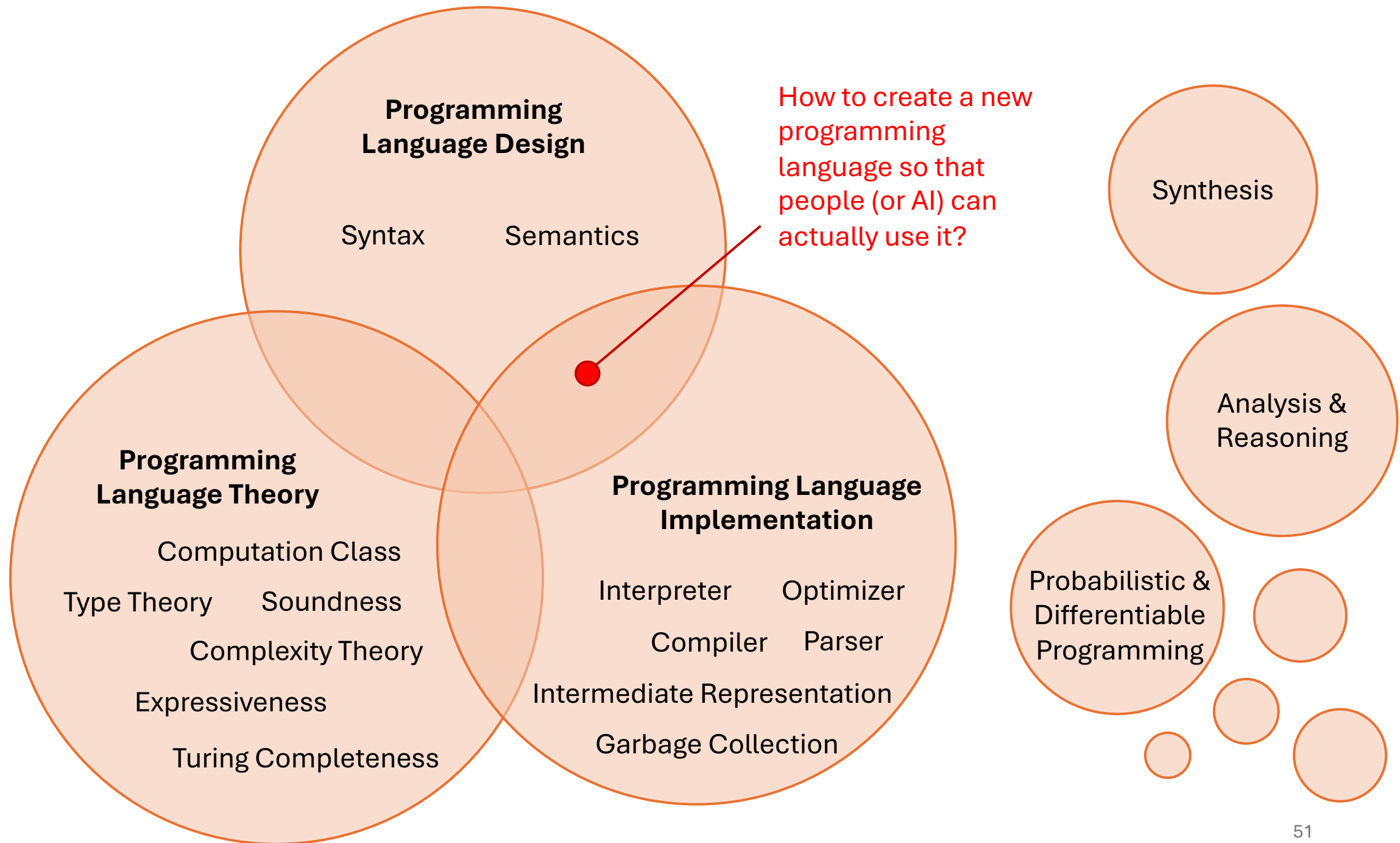
Copies from register $\bar{s}_n$, truncating if the destination is smaller.
 Lexicographically sorts the table with columns $\bar{b}_n$ and $\bar{a}_n$.
 Merges adjacent duplicate rows via σ from the unique elements s .
 Merges two sorted tables with columns $\bar{a}_n$ and $\bar{b}_n$.

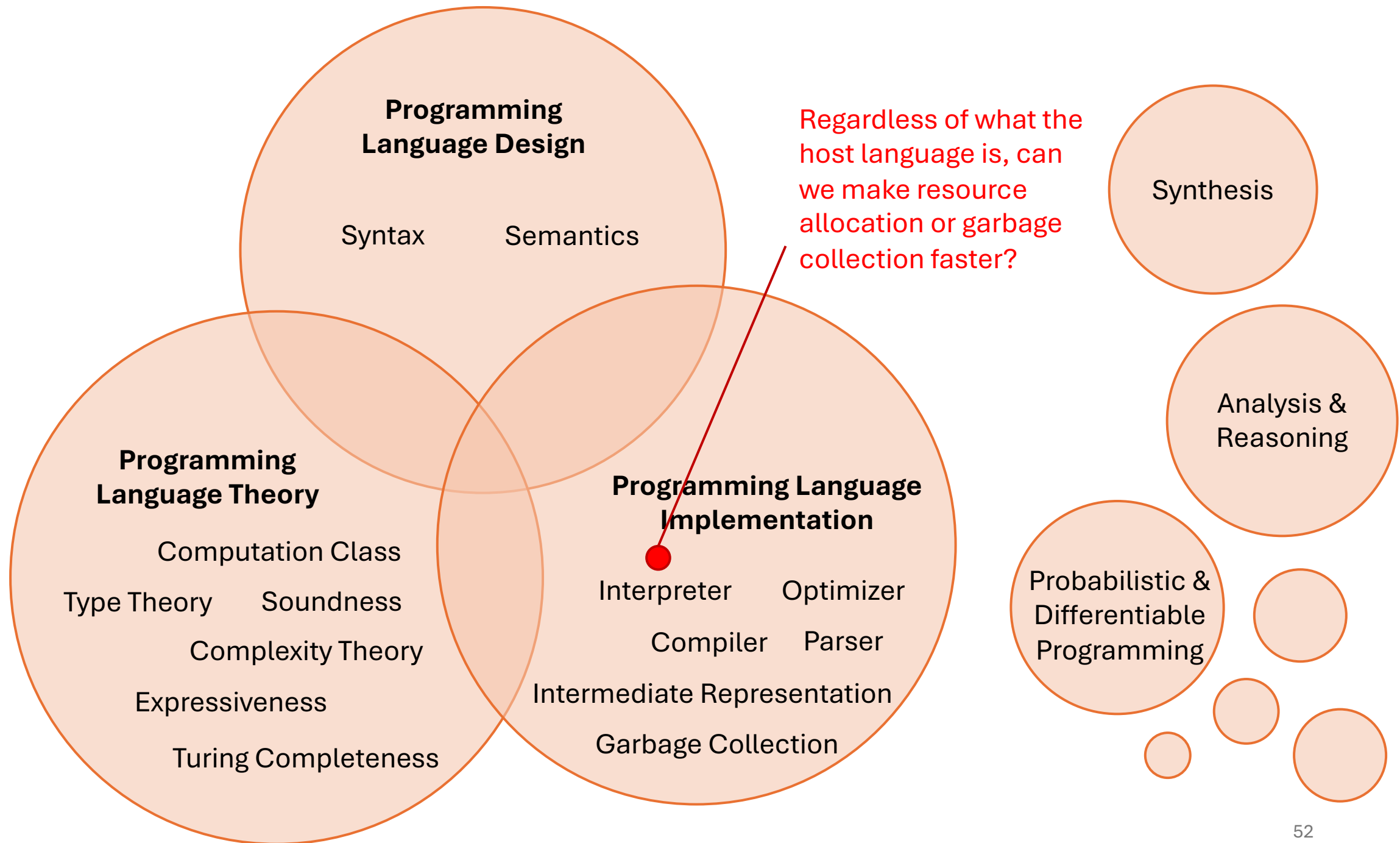
Lobster

Scallop

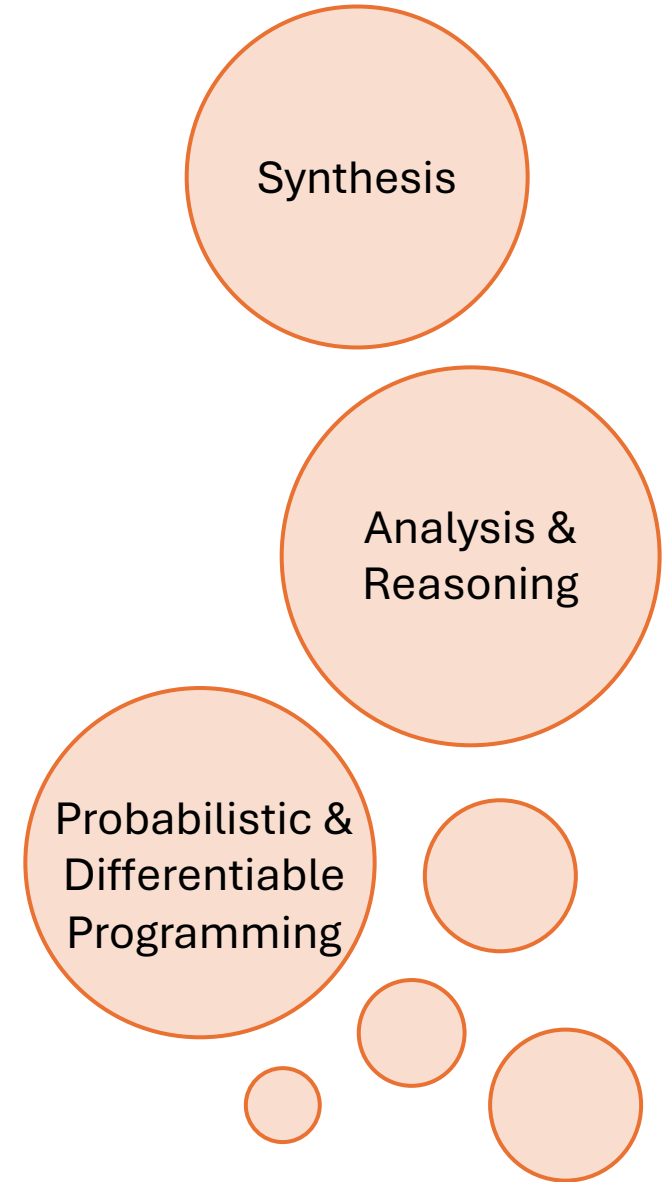
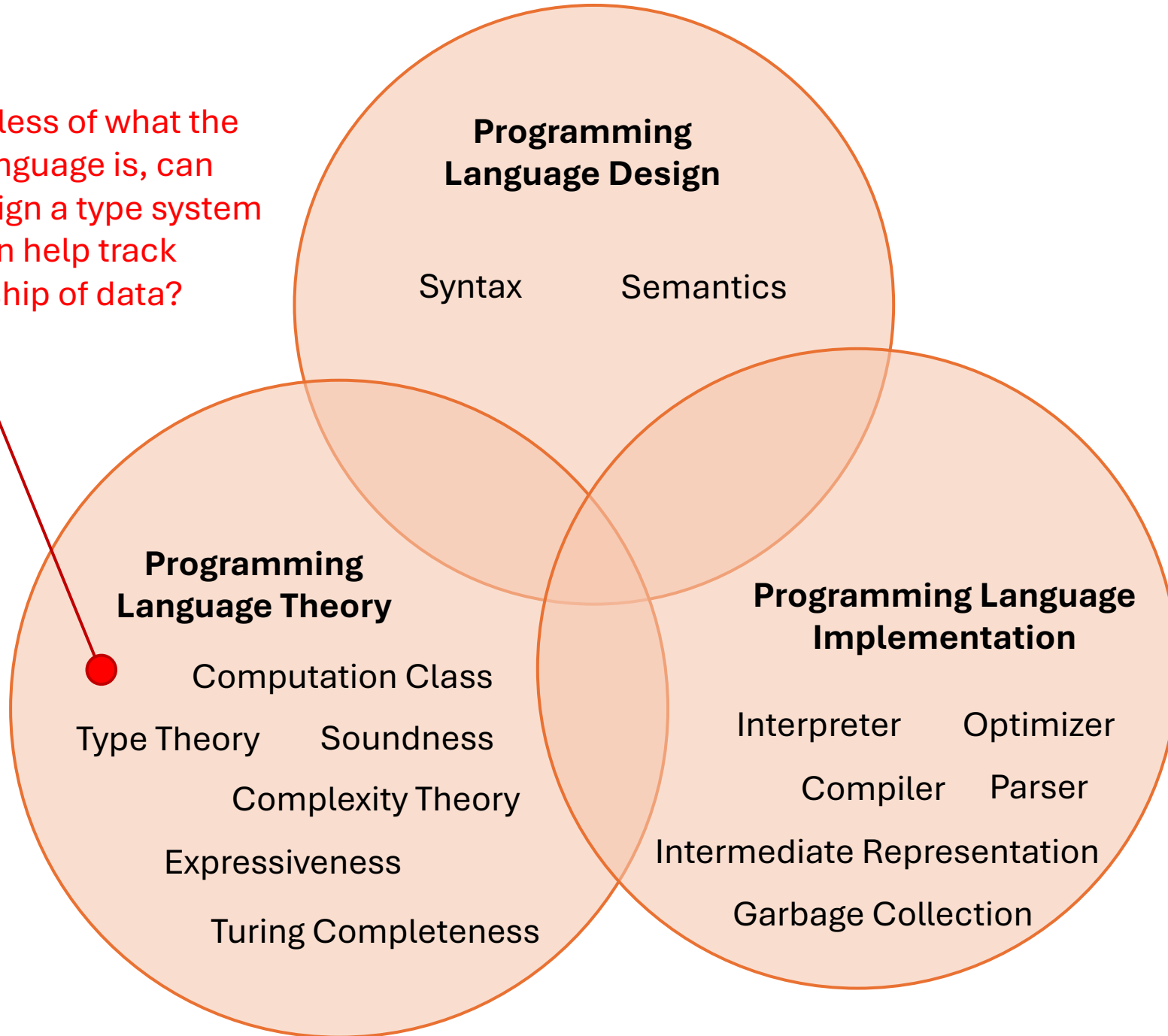




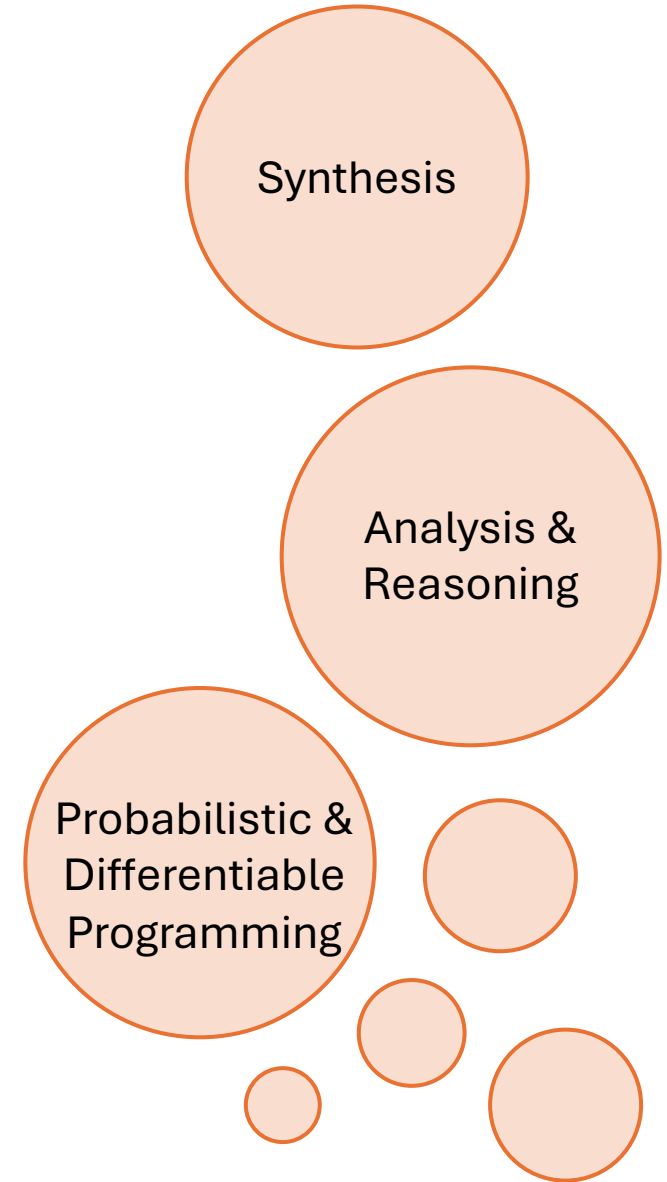
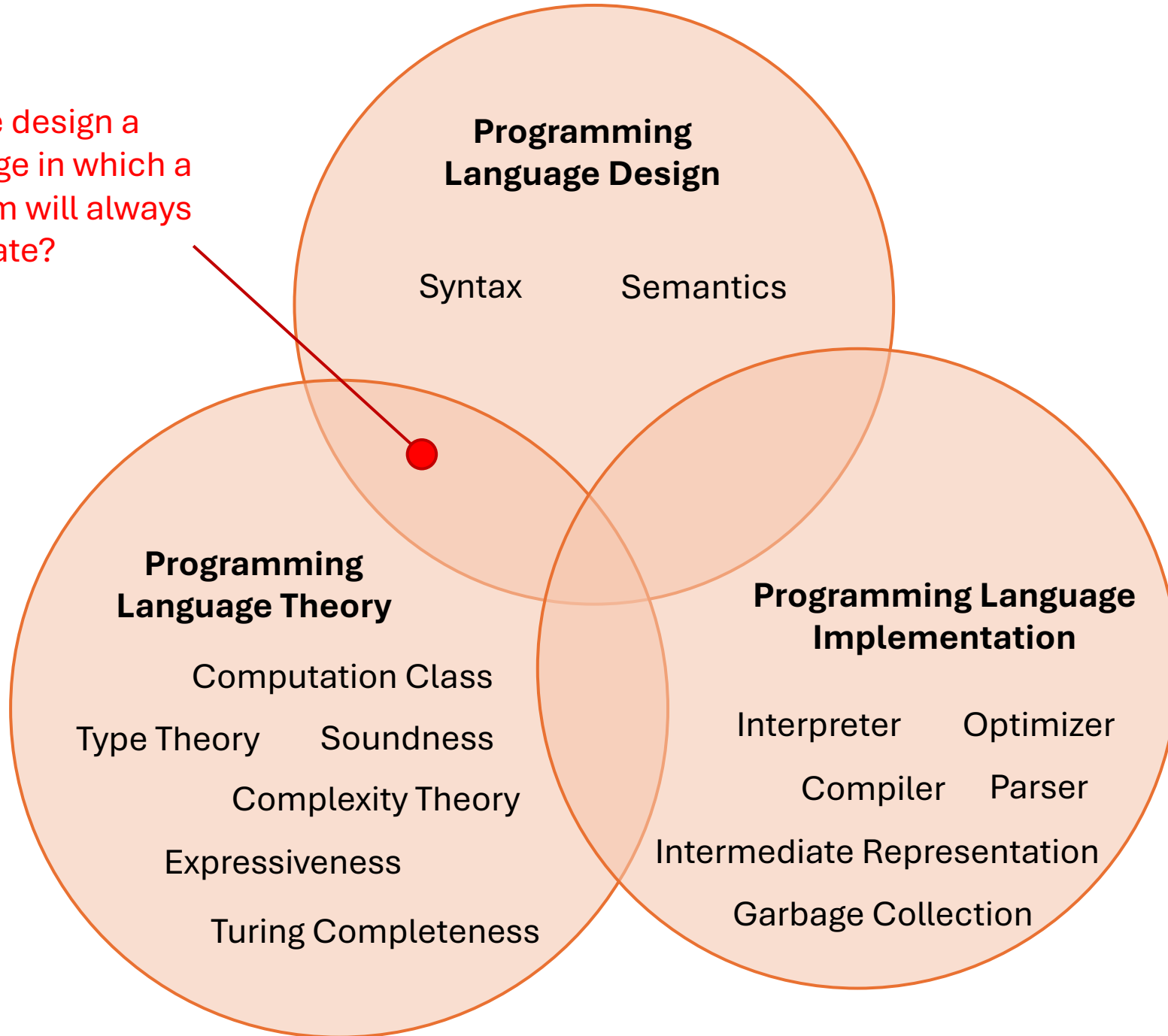


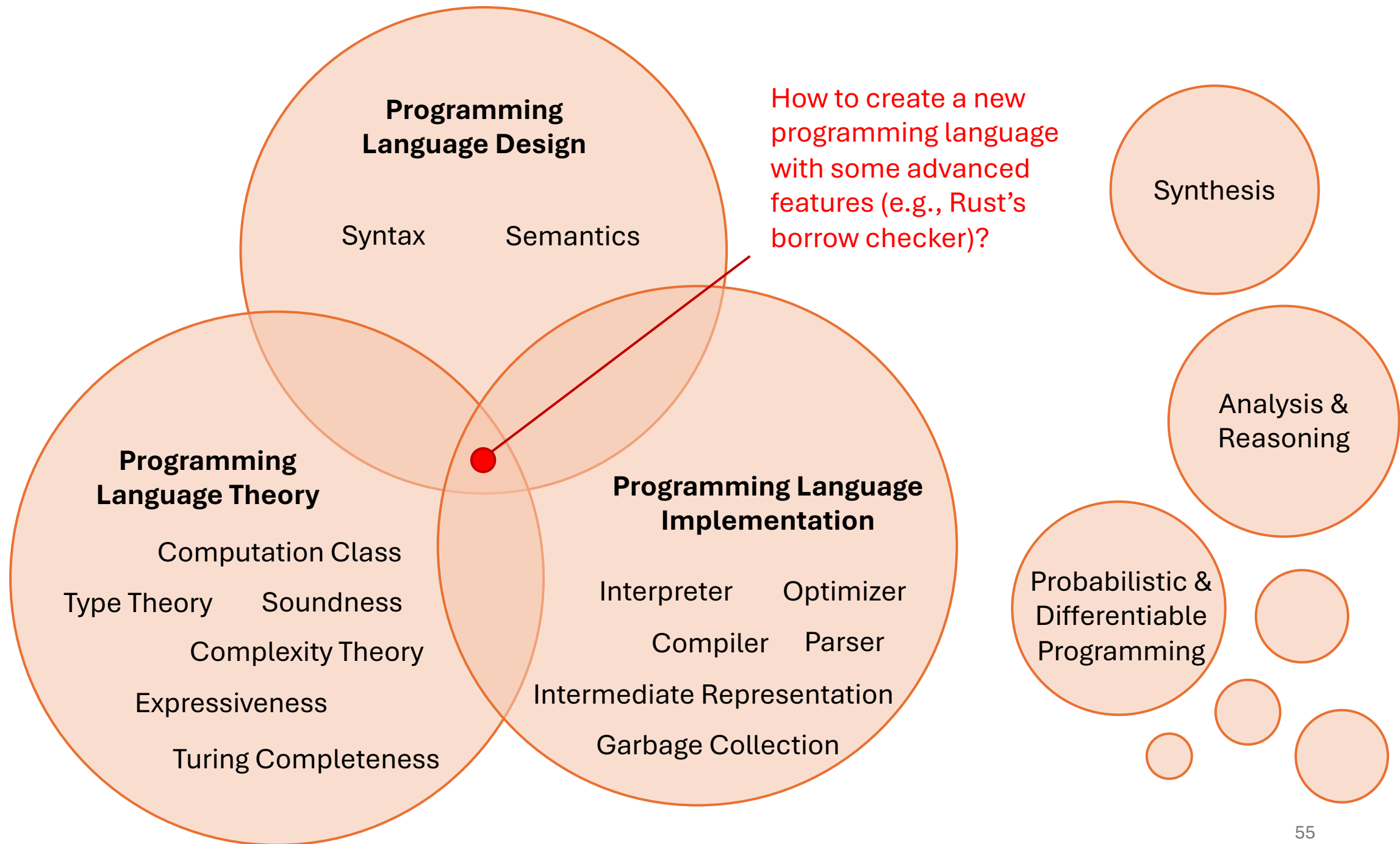


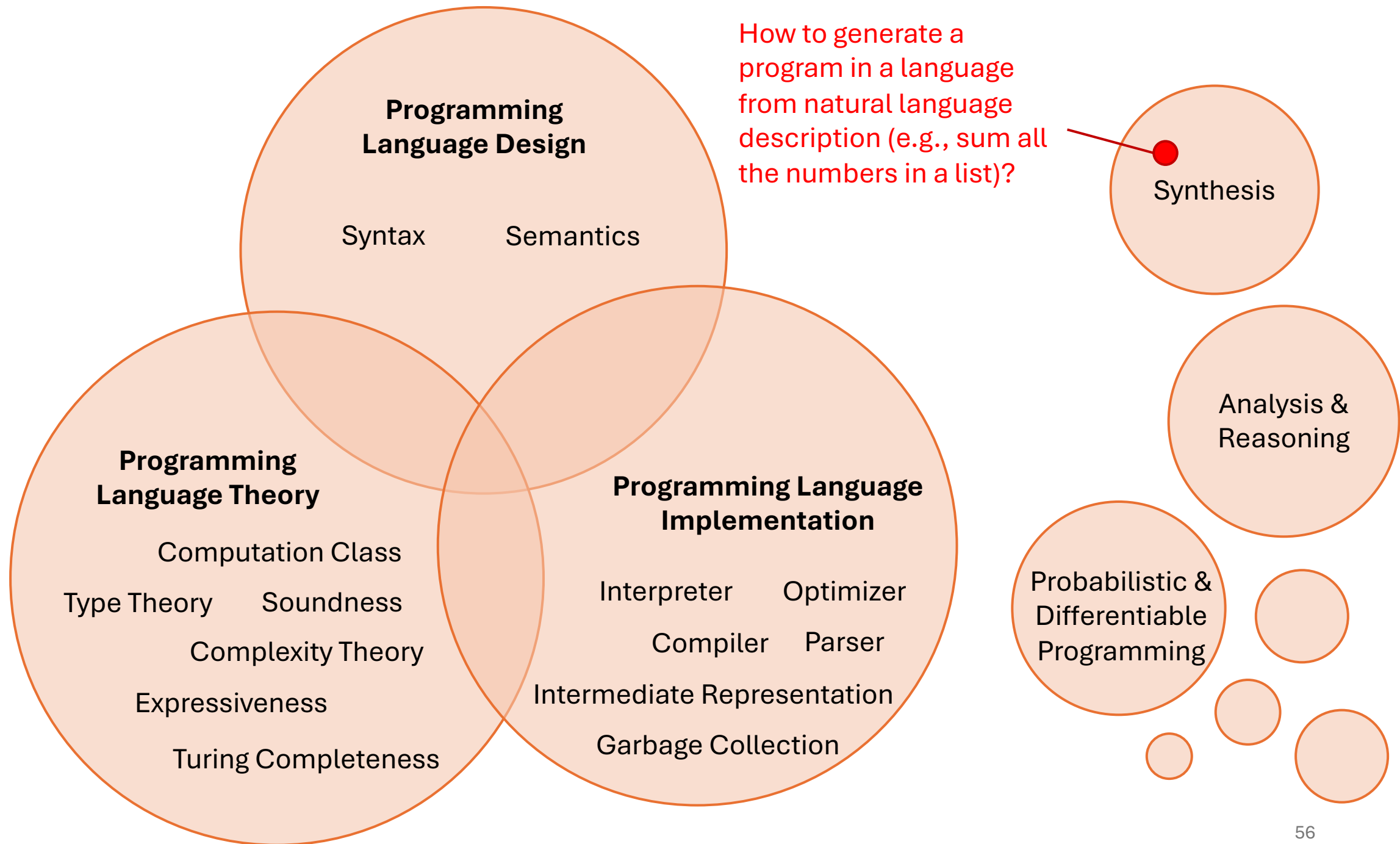
Regardless of what the host language is, can we design a type system that can help track ownership of data?

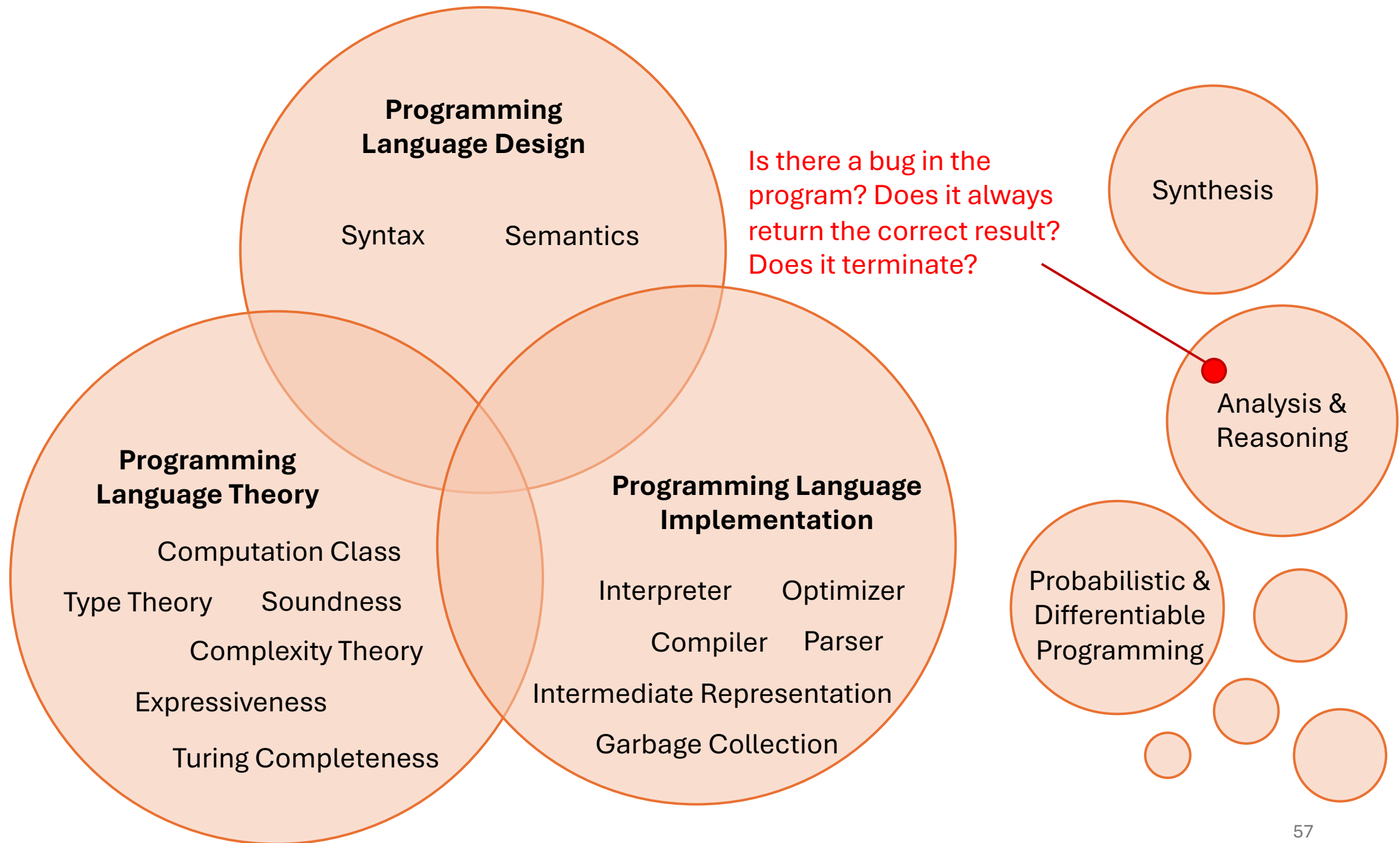


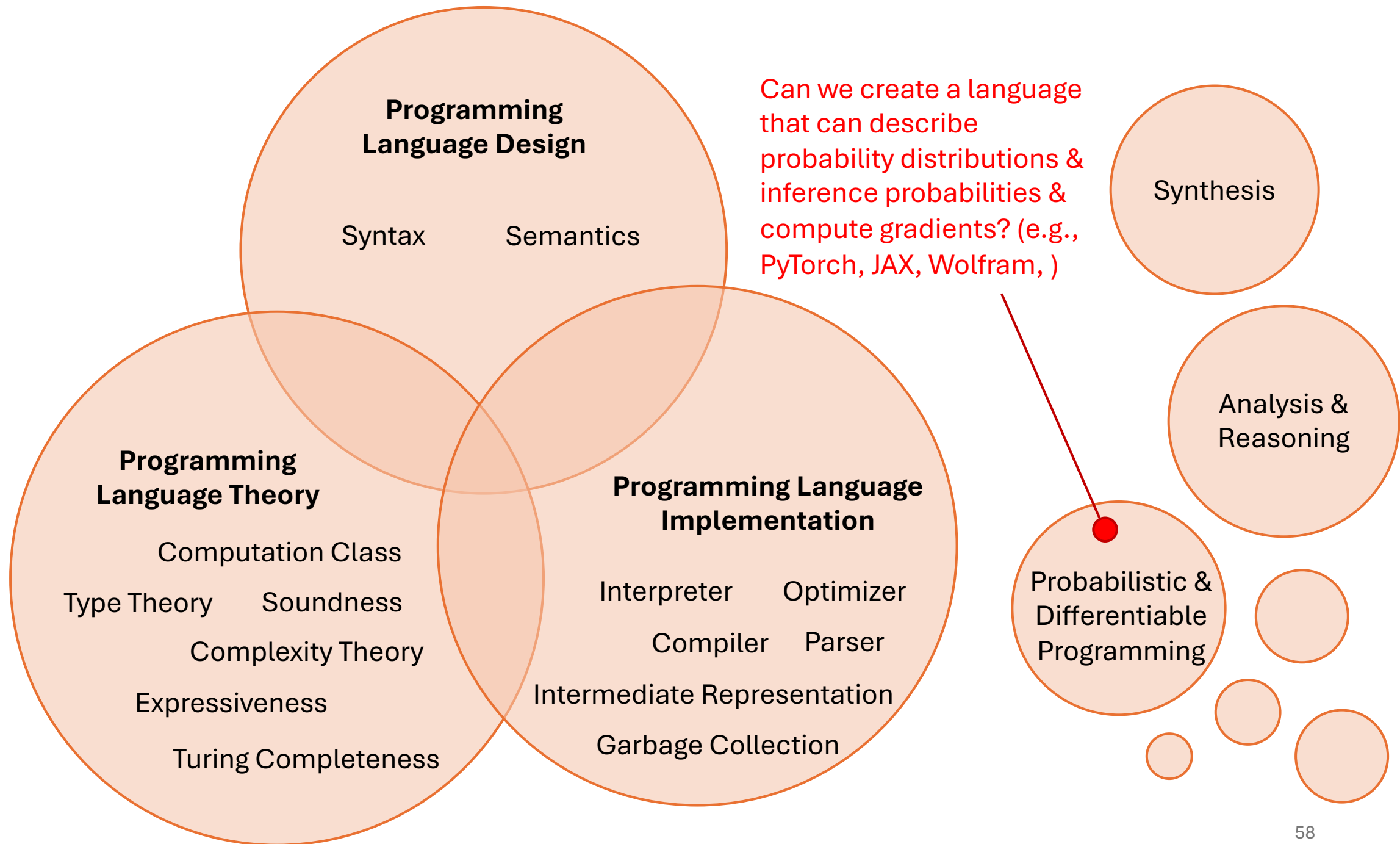
Can we design a language in which a program will always terminate?

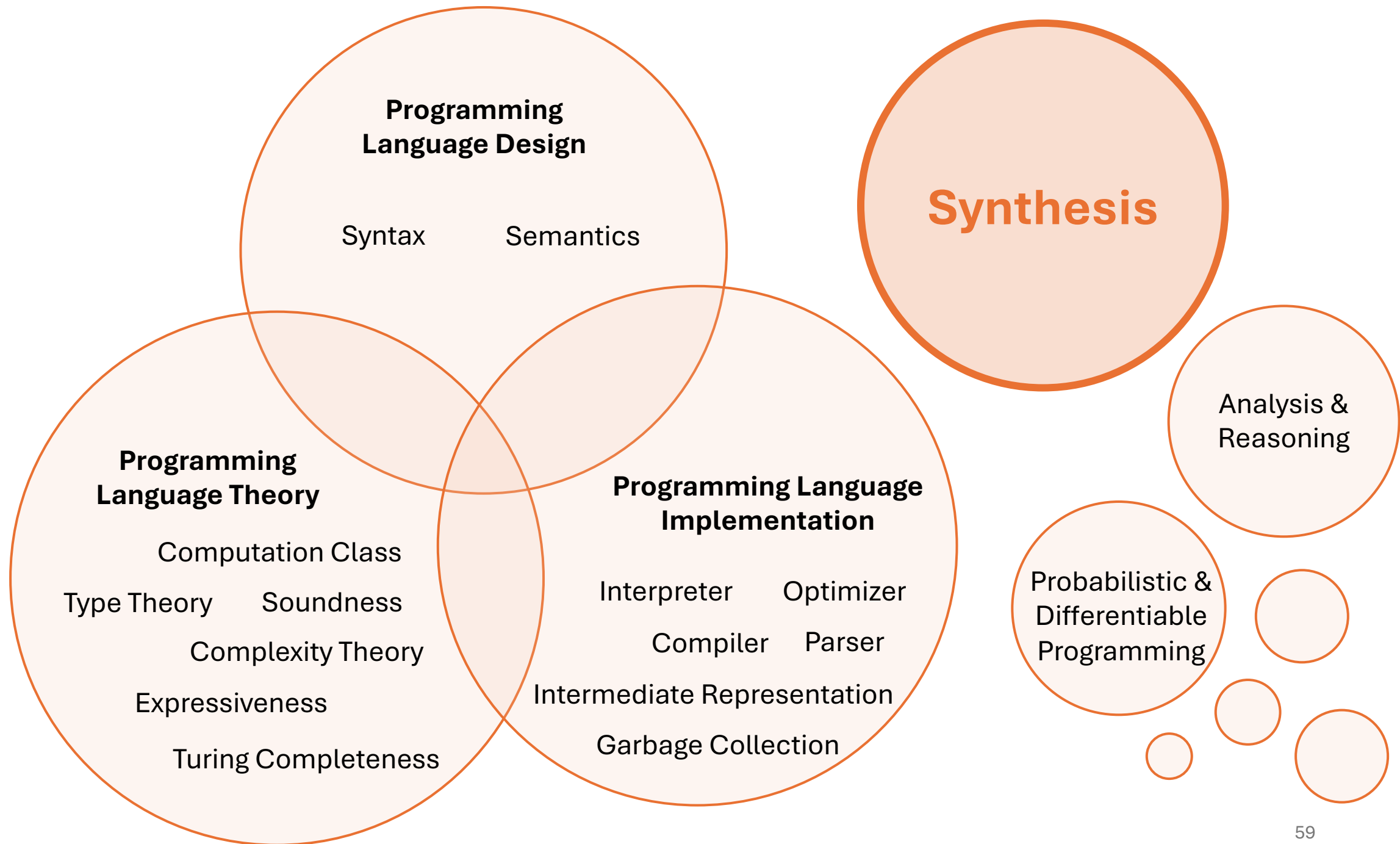


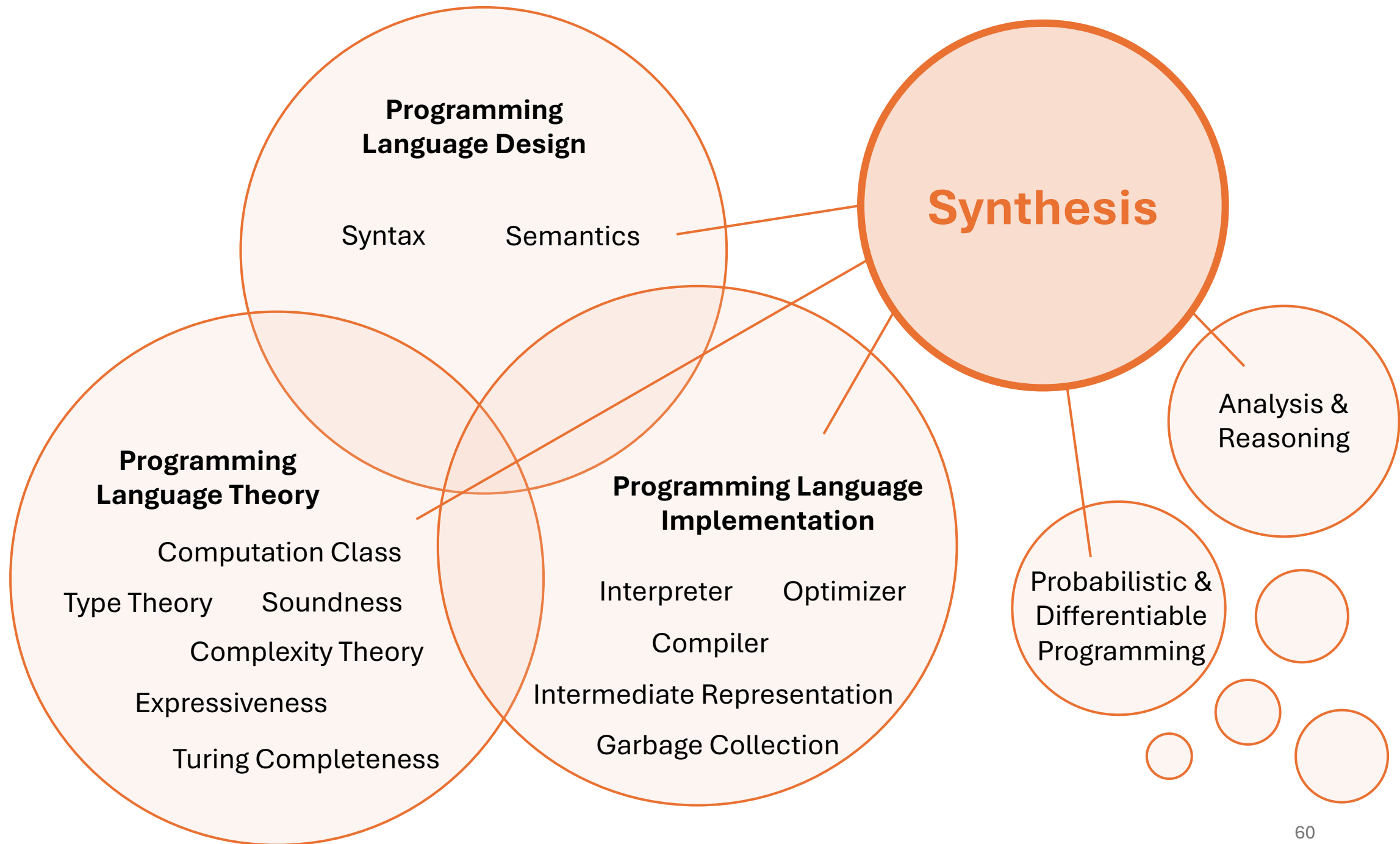












Grading

- Attendance (10%)
- Assignment 1: Inductive synthesis (15%)
- Assignment 2: Evaluating coding LLMs (15%)
- Assignment 3: Coding agents (15%)
- Oral presentation (10%)
- Final Project (35%)

Assignments (45% total, 15% each)

- There will be **three assignments**:
 - Assignment 1: Inductive Synthesis & Basics of LLM synthesis
 - Assignment 2: Evaluating Coding LLMs
 - Assignment 3: Coding Agents
- Timeline: **~2 weeks** for each assignment
- Submission: via **GradeScope**; submit .zip files
- Late submission: up to 3 days, -10% per day
- Extensions: possible, please request via email

Assignments (45% total, 15% each)

- Collaboration policy:
 - **Encouraged**, please acknowledge your collaborator
 - Still need to be your own work
- Use of AI:
 - **Encouraged**, please acknowledge your collaborator AI
 - Be specific about the model, version, programming environment, etc.
 - Document important conversations, learn about its good or bad

Assignments (45% total, 15% each)

- **API Key** for LLM use
 - Assignments will include code for you to invoke LLMs, which require API keys
 - You will use Amazon Bedrock API Keys (\$200 free credits after registration)
 - A tutorial will be sent to all of you so that you can gain access
 - This should be sufficient for doing the assignments

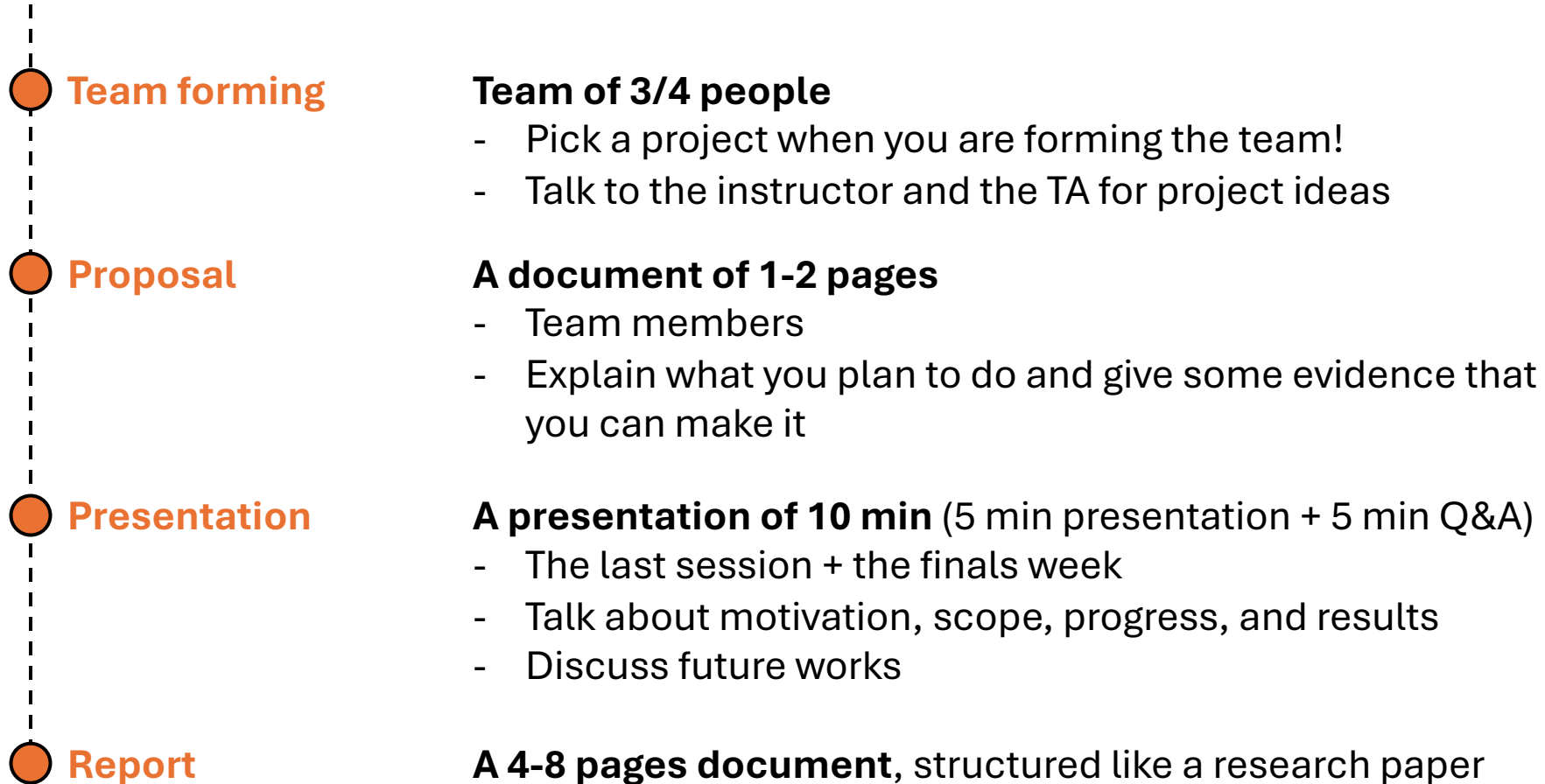
Oral Presentation (10%)

- Happening on the latter half of the semester
- **Six** lectures will be dedicated to oral presentations by all of you
- There are going to be six topics selected by a poll, for each topic
 - We assign 3-4 papers, for each paper
 - Two people are going to lead the discussion of that paper
 - One person is going to be the “**Defender**” – presenting the main content
 - Another person is going to be the “**Attacker**” – asking the questions like a reviewer
 - 10 minutes defending, 10 minutes attacking
- Graded on a completion basis
- The schedule is going to be done via Google Form
 - Slots are first-come-first-serve

Final Project

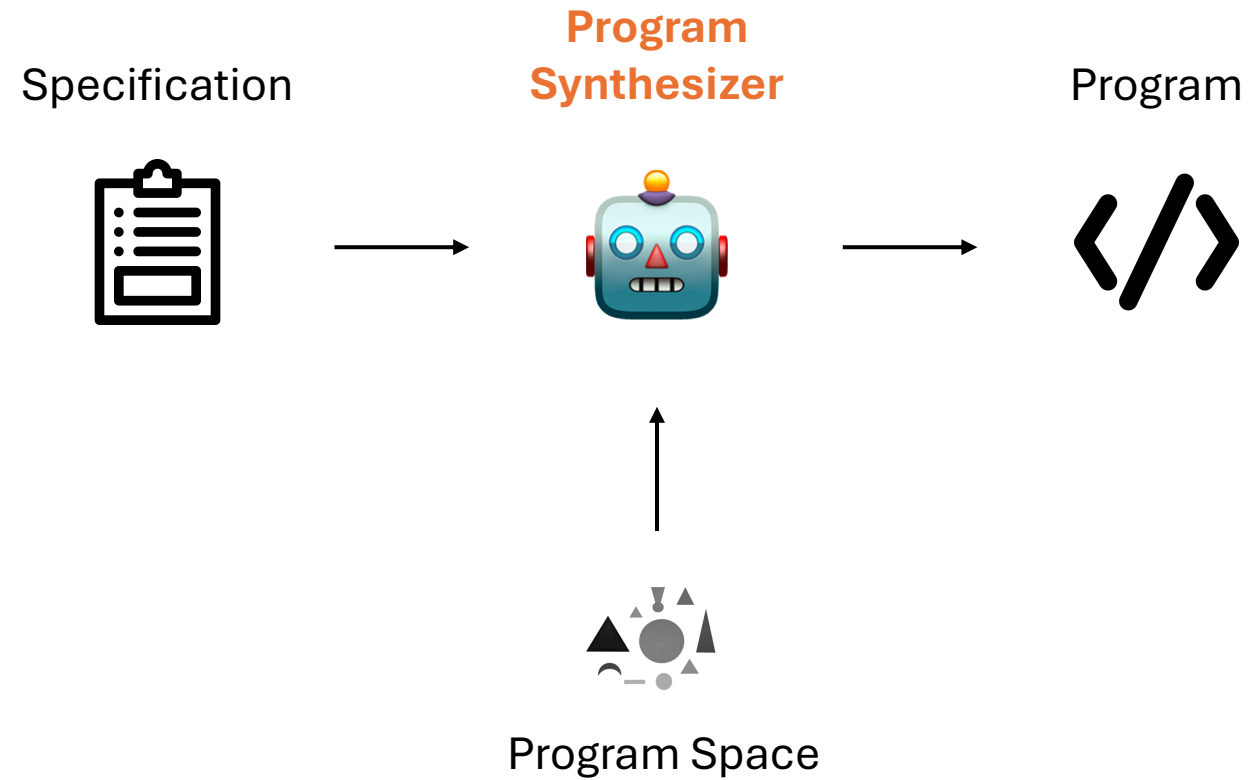
- On any aforementioned topics:
 - Re-implement a technique from a paper
 - Applying existing technique to a new domain
 - Extend/improve existing synthesis algorithm or tool
 - Develop a new synthesis technique or tool
 - New dataset, new benchmark, or a novel evaluation
 - ...
- Judged in terms of
 - Quality of execution
 - Originality
 - Scope

Final Project

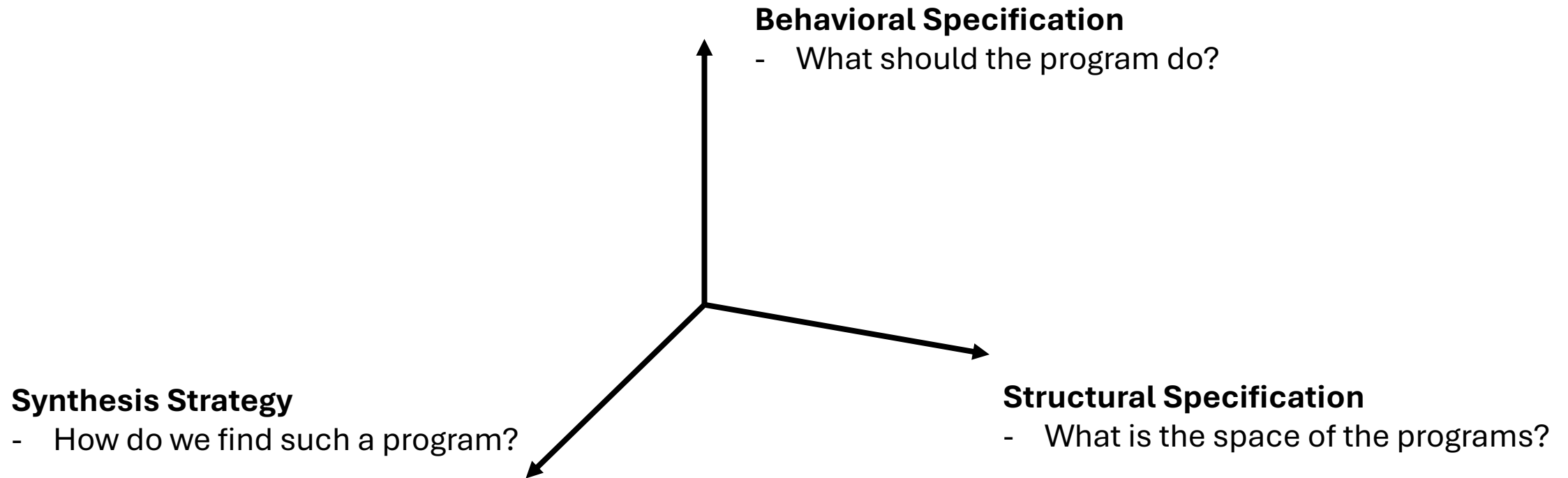


Program Synthesis

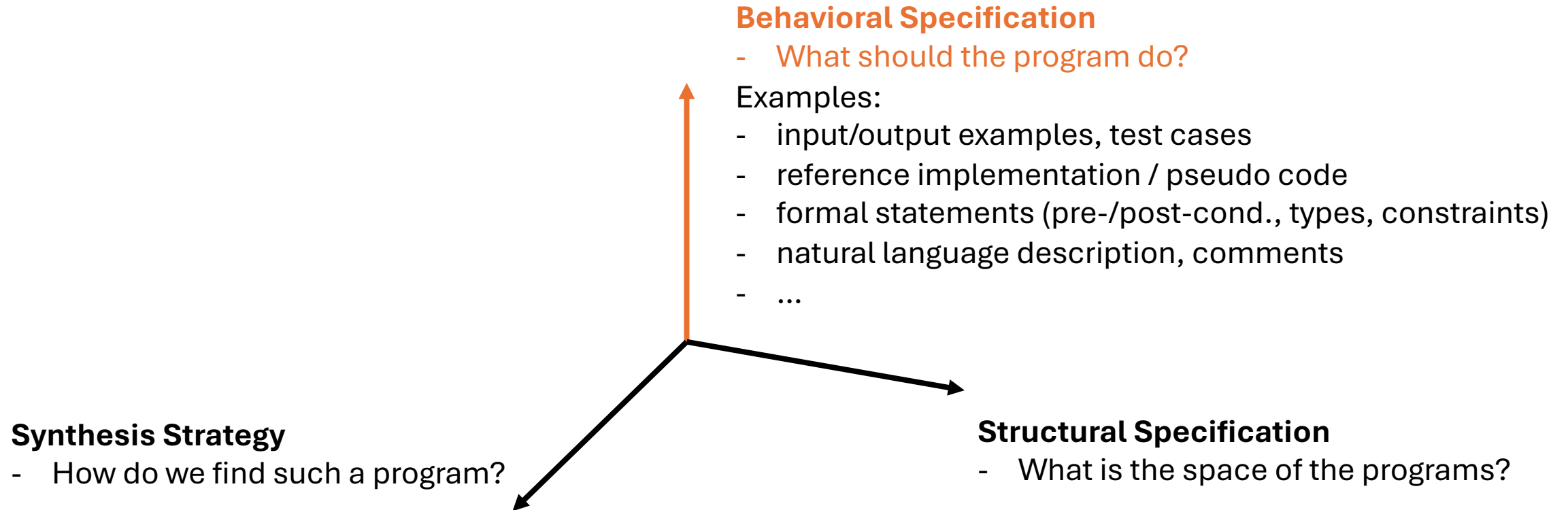
Program Synthesizers



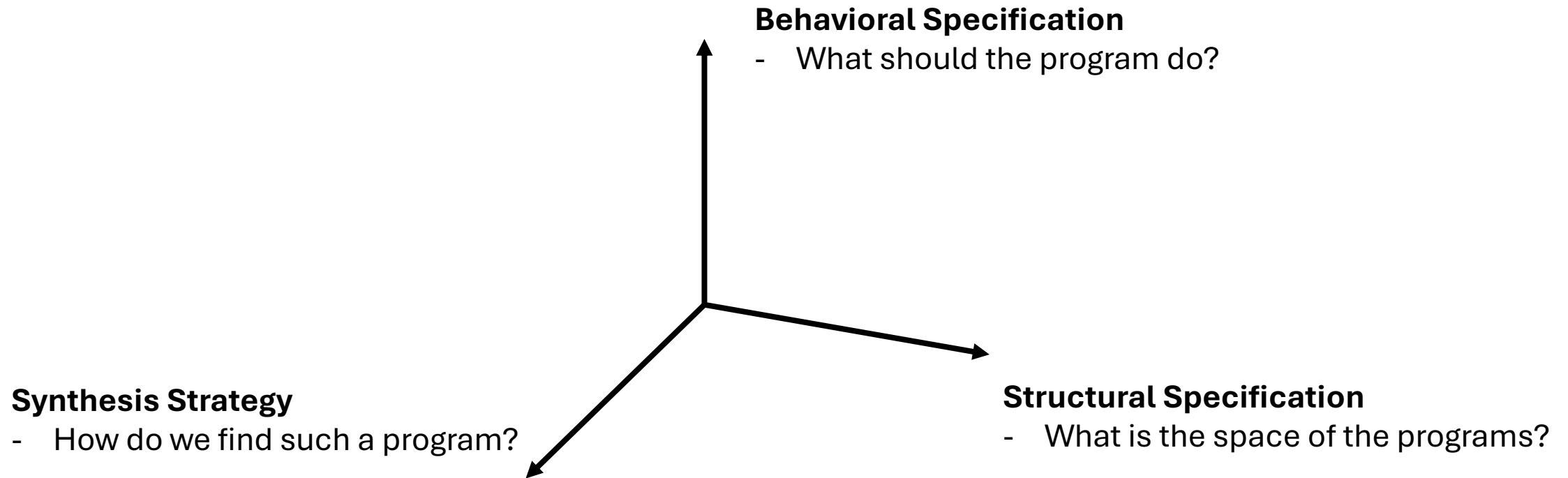
Dimensions in Program Synthesis



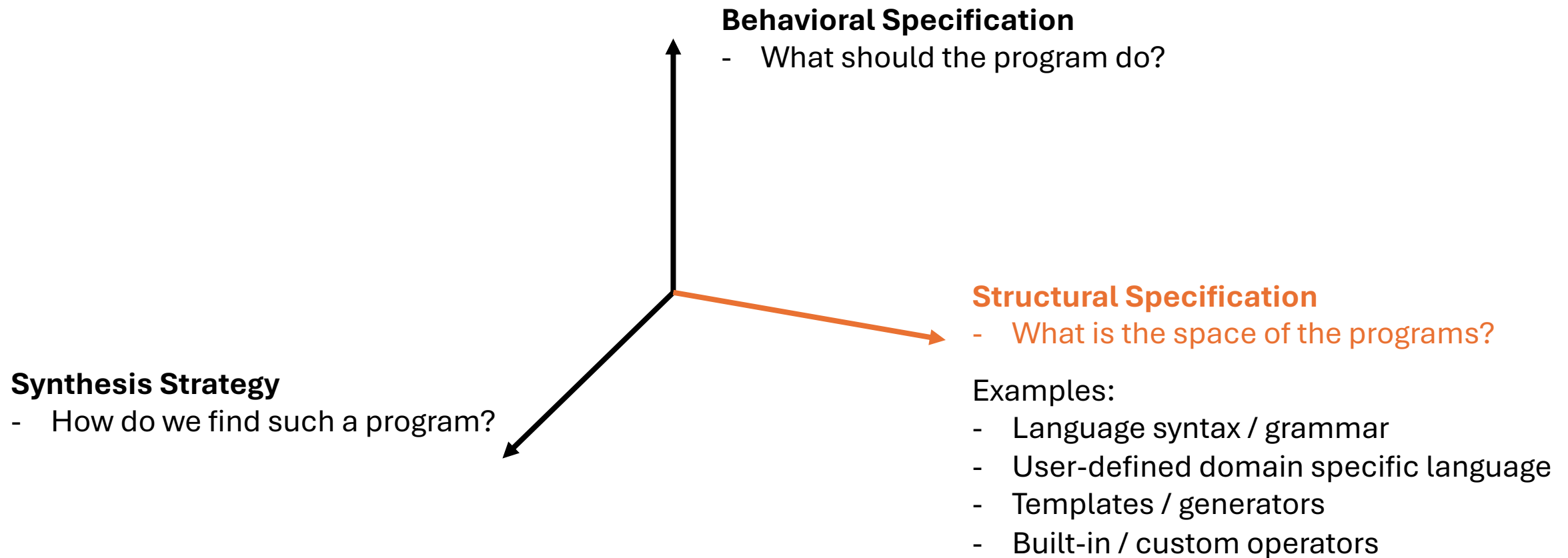
Dimensions in Program Synthesis



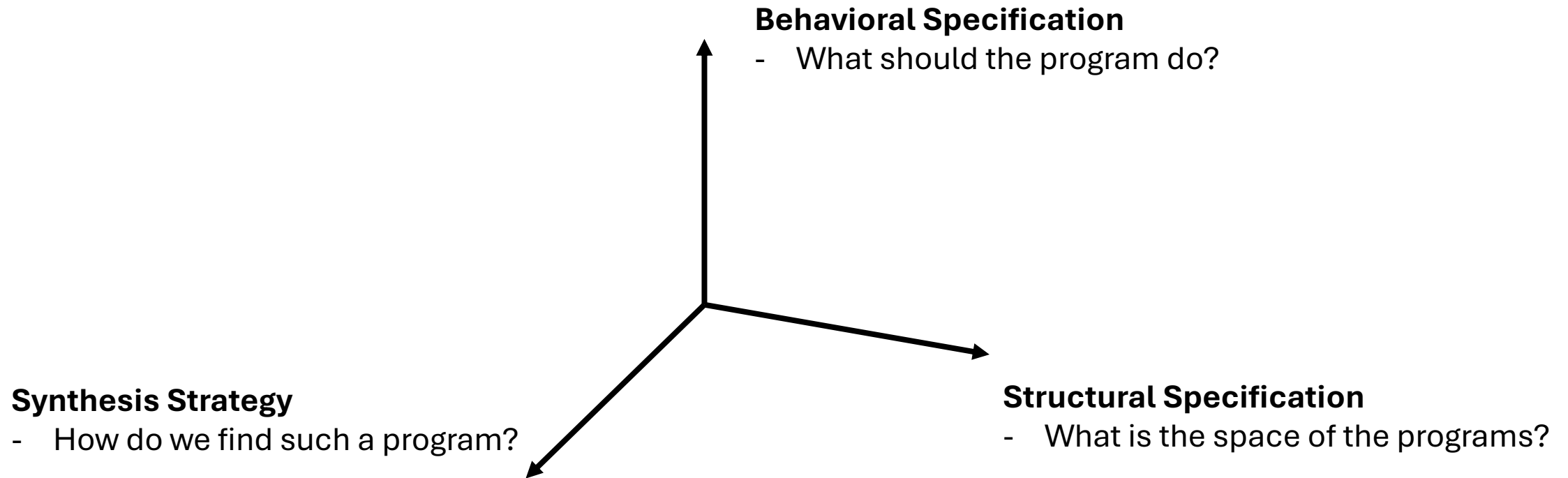
Dimensions in Program Synthesis



Dimensions in Program Synthesis



Dimensions in Program Synthesis



Dimensions in Program Synthesis

Behavioral Specification

- What should the program do?

Synthesis Strategy

- How do we find such a program?

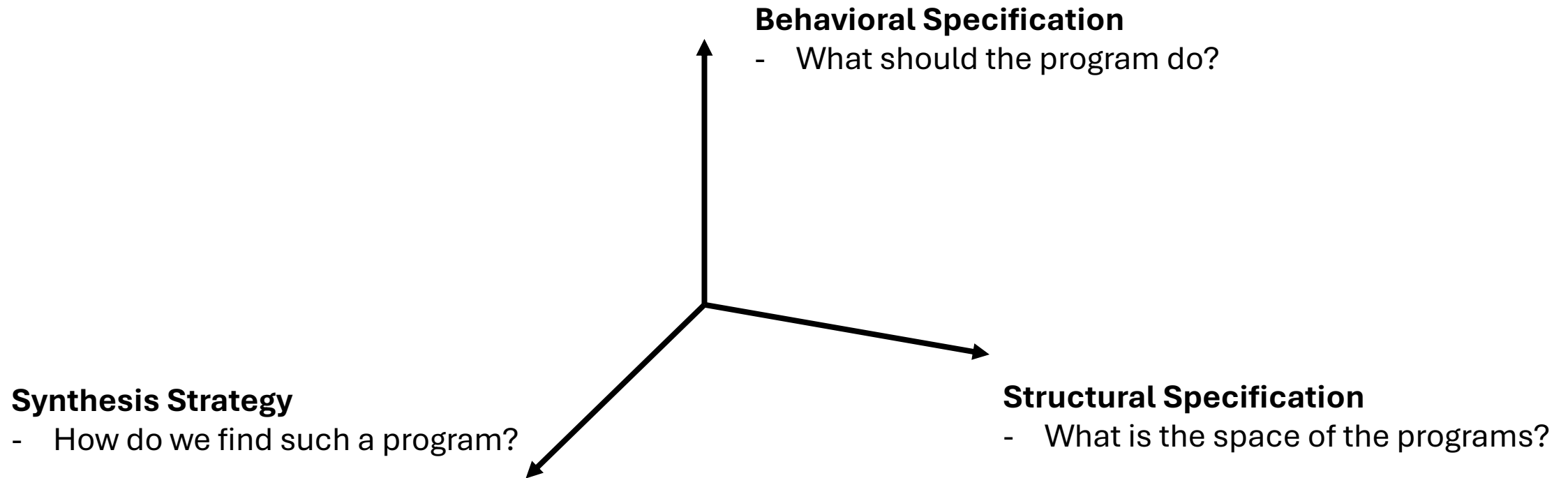
Examples:

- Bottom-up / top-down search
- Stochastic / constraint-based search
- LLM zero-shot / few-shot prompting
- Chain-of-thought reasoning
- Iterative / agentic self-refinement

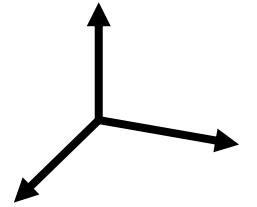
Structural Specification

- What is the space of the programs?

Dimensions in Program Synthesis



Program Synthesis in Action



Behavioral Specification

- What should the program do?

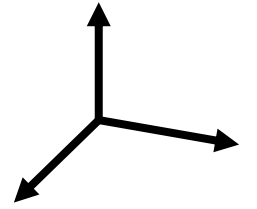
Structural Specification

- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?

Program Synthesis in Action



Behavioral Specification

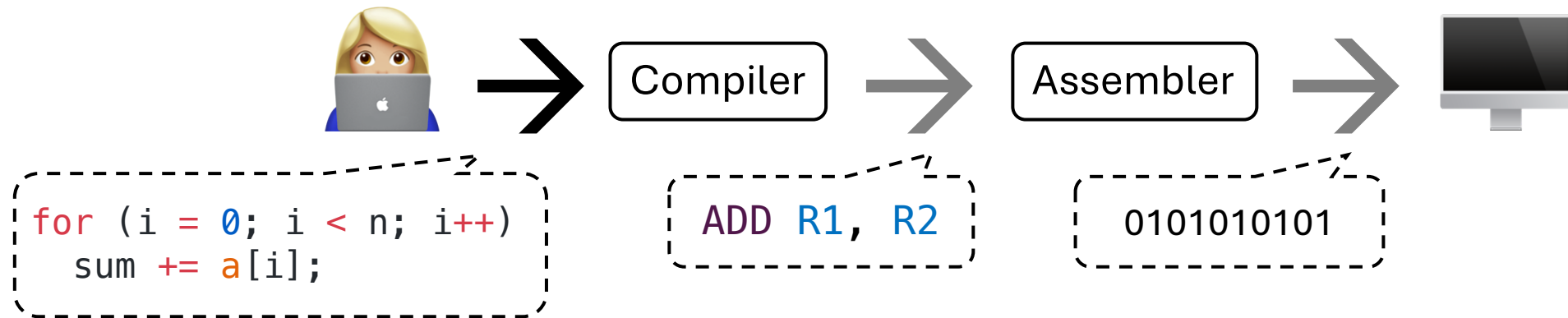
- What should the program do?

Structural Specification

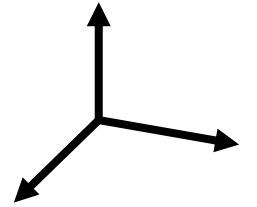
- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?



Program Synthesis in Action



Behavioral Specification

- What should the program do?

A program in high-level programming language (Fortran, C, Rust, etc.)

Structural Specification

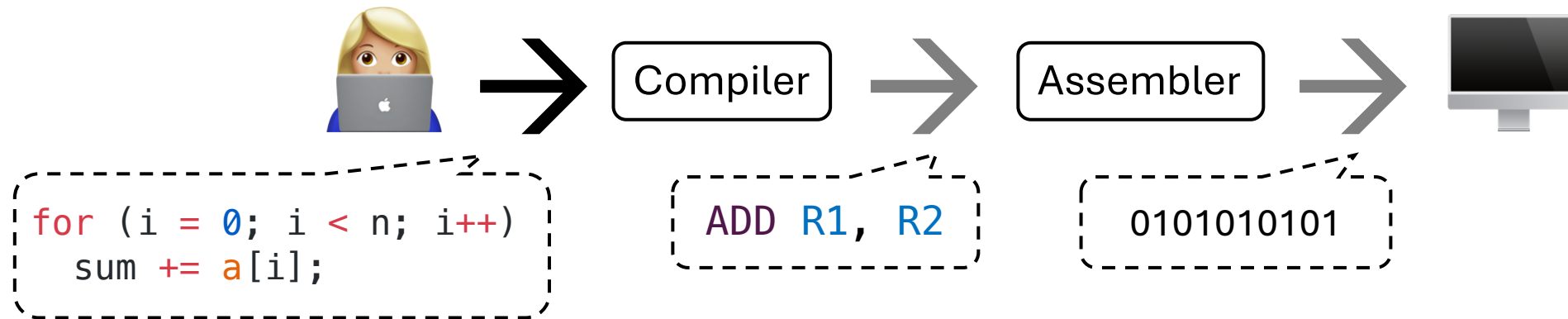
- What is the space of the programs?

A program in binary machine code

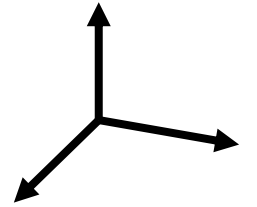
Synthesis Strategy

- How do we find such a program?

Compilation, Optimization, Assembling, etc.



Program Synthesis in Action



Behavioral Specification

- What should the program do?

Structural Specification

- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?

Quick Fill Auto Fill Quick Layout

Apply HiLight CurrencyWidget

Undo Commit AddressWidget

Table116		Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171					
	A	B	C	D	E	F	
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6	
2	Ana Trujillo	357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064	27171
3	Antonio Moreno	515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581					
4	Thomas Hardy	742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607					
5	Christina Berglund	475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146					
6	Hanna Moos	785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284					
7	Frédérique Citeaux	308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415					
8	Martin Sommer	887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536					
9	Laurence Lebihan	944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,25252					
10	Elizabeth Lincoln	452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279					
11	Victoria Ashworth	463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832					
12	Patricio Simpson	630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525					
13	Francisco Chang	683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453					
14	Yang Wang	944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388					
15	Pedro Afonso	411 70th Place ,Kent,WA,(170) 555-2964,774-35-2298,29485					
16	Elizabeth Brown	971 20th Lane ,Puyallup,WA,(373) 555-4134,476-53-7164,26417					
17	Sven Ottlieb	676 17th Lane NE,Redmond,WA,(828) 555-1593,548-73-8633,27440					
18	Janine Labrune	267 95th Place SE,Seattle,WA,(949) 555-1316,350-27-8300,28074					
19	Ann Devon	694 53th Place ,Kent,WA,(194) 555-8124,559-74-4016,22367					
20	Roland Mendel	581 12th Street NW,Kent,WA,(103) 555-2146,303-79-1328,20518					
21	Aria Cruz	594 85th Lane ,Renton,WA,(431) 555-1376,329-93-9992,21498					
22	Diego Roel	550 22th Lane ,Renton,WA,(639) 555-6238,918-34-5172,25931					
23	Martine Rancé	688 93th Place NW,Kent,WA,(573) 555-3571,695-94-3479,22424					
24							
25							
26							

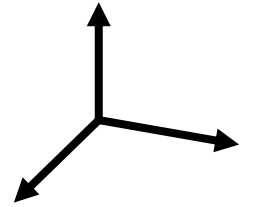
Quick Fill Auto Fill Quick Layout

Apply HiLight CurrencyWidget

Undo Commit AddressWidget

Table116 Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171								
	A	B	C	D	E	F		
1	Column1	Col 2	Col 3	Col 4	Col 5	Col 6		
2	Ana Trujillo 357 21th Place SE,Redmond,WA,(757) 555-1634,140-37-6064,27171	Redmond	WA	(757) 555-1634	140-37-6064	27171		
3	Antonio Moreno 515 93th Lane ,Renton,WA,(411) 555-2786,562-87-3127,28581	Renton	WA	(411) 555-2786	562-87-3127	28581		
4	Thomas Hardy 742 17th Street NE,Seattle,WA,(412) 555-5719,921-29-4931,24607	Seattle	WA	(412) 555-5719	921-29-4931	24607		
5	Christina Berglund 475 22th Lane ,Redmond,WA,(443) 555-6774,844-35-6764,30146	Redmond	WA	(443) 555-6774	844-35-6764	30146		
6	Hanna Moos 785 45th Street NE,Puyallup,WA,(376) 555-2462,515-68-1285,29284	Puyallup	WA	(376) 555-2462	515-68-1285	29284		
7	Frédérique Citeaux 308 66th Place ,Redmond,WA,(689) 555-2770,552-23-2508,21415	Redmond	WA	(689) 555-2770	552-23-2508	21415		
8	Martin Sommer 887 86th Place ,Kent,WA,(715) 555-5450,870-91-9824,21536	Kent	WA	(715) 555-5450	870-91-9824	21536		
9	Laurence Lebihan 944 13th Street NE,Redmond,WA,(620) 555-2361,649-25-5312,2525	Redmond	WA	(620) 555-2361	649-25-5312	25252		
10	Elizabeth Lincoln 452 73th Lane NE,Renton,WA,(851) 555-4561,425-97-6344,22279	Renton	WA	(851) 555-4561	425-97-6344	22279		
11	Victoria Ashworth 463 16th Street ,Renton,WA,(696) 555-6044,690-29-7926,22832	Renton	WA	(696) 555-6044	690-29-7926	22832		
12	Patricio Simpson 630 20th Street ,Redmond,WA,(179) 555-3265,389-78-3236,24525	Redmond	WA	(179) 555-3265	389-78-3236	24525		
13	Francisco Chang 683 49th Lane ,Seattle,WA,(272) 555-7434,665-18-6435,29453	Seattle	WA	(272) 555-7434	665-18-6435	29453		
14	Yang Wang 944 28th Lane ,Redmond,WA,(151) 555-2272,846-78-8452,24388	Redmond	WA	(151) 555-2272	846-78-8452	24388		
15	Pedro Afonso 411 70th Place ,Kent,WA,(170) 555-2964,774-35-2298,29485	Kent	WA	(170) 555-2964	774-35-2298	29485		
16	Elizabeth Brown 971 20th Lane ,Puyallup,WA,(373) 555-4134,476-53-7164,26417	Puyallup	WA	(373) 555-4134	476-53-7164	26417		
17	Sven Ottlieb 676 17th Lane NE,Redmond,WA,(828) 555-1593,548-73-8633,27440	Redmond	WA	(828) 555-1593	548-73-8633	27440		
18	Janine Labrune 267 95th Place SE,Seattle,WA,(949) 555-1316,350-27-8300,28074	Seattle	WA	(949) 555-1316	350-27-8300	28074		
19	Ann Devon 694 53th Place ,Kent,WA,(194) 555-8124,559-74-4016,22367	Kent	WA	(194) 555-8124	559-74-4016	22367		
20	Roland Mendel 581 12th Street NW,Kent,WA,(103) 555-2146,303-79-1328,20518	Kent	WA	(103) 555-2146	303-79-1328	20518		
21	Aria Cruz 594 85th Lane ,Renton,WA,(431) 555-1376,329-93-9992,21498	Renton	WA	(431) 555-1376	329-93-9992	21498		
22	Diego Roel 550 22th Lane ,Renton,WA,(639) 555-6238,918-34-5172,25931	Renton	WA	(639) 555-6238	918-34-5172	25931		
23	Martine Rancé 688 93th Place NW,Kent,WA,(573) 555-3571,695-94-3479,22424	Kent	WA	(573) 555-3571	695-94-3479	22424		
24								
25								
26								

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Fit the user demonstrated
example rows

Structural Specification

- What is the space of the programs?

Excel formula language

Synthesis Strategy

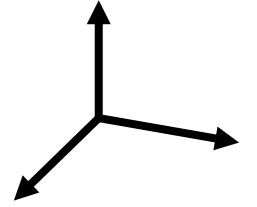
- How do we find such a program?

Programming by
demonstration (PbD)

Column1	Column2	Column3	Column4	Column5	Column6
Ana Trujillo	357 21th Place SE, Redmond, WA, (757) 555-1634, 140-37-6064, 27171	Redmond	WA	(757) 555-1634	140-37-6064 27171
Antonio Moreno	515 93th Lane, Renton, WA, (411) 555-2786, 562-87-3127, 28581	Renton	WA	(411) 555-2786	562-87-3127 28581
Thomas Hardy	742 17th Street NE, Seattle, WA, (412) 555-5719, 921-29-4931, 24607	Seattle	WA	(412) 555-5719	921-29-4931 24607
Christina Berglund	475 22th Lane, Redmond, WA, (443) 555-6774, 844-35-6764, 30146	Redmond	WA	(443) 555-6774	844-35-6764 30146
Hanna Moos	785 45th Street NE, Puyallup, WA, (376) 555-2462, 515-68-1285, 29284	Puyallup	WA	(376) 555-2462	515-68-1285 29284
Frederique Citeaux	308 66th Place, Redmond, WA, (689) 555-2770, 552-23-2508, 21415	Redmond	WA	(689) 555-2770	552-23-2508 21415
Martin Sommer	887 86th Place, Kent, WA, (715) 555-5450, 870-91-9824, 21536	Kent	WA	(715) 555-5450	870-91-9824 21536
Laurence Leblan	944 13th Street NE, Redmond, WA, (620) 555-2361, 649-25-5312, 25252	Redmond	WA	(620) 555-2361	649-25-5312 25252
Elizabeth Lincoln	452 73th Lane NE, Renton, WA, (851) 555-4561, 425-97-6344, 22279	Renton	WA	(851) 555-4561	425-97-6344 22279
Victoria Ashworth	463 16th Street, Renton, WA, (696) 555-6044, 690-29-7926, 22832	Renton	WA	(696) 555-6044	690-29-7926 22832
Patricio Simpson	630 20th Street, Redmond, WA, (179) 555-3265, 389-78-3236, 24525	Redmond	WA	(179) 555-3265	389-78-3236 24525
Francisco Chang	683 49th Lane, Seattle, WA, (272) 555-7434, 665-18-6435, 29453	Seattle	WA	(272) 555-7434	665-18-6435 29453
Yang Wang	944 28th Lane, Redmond, WA, (151) 555-2272, 846-78-8452, 24388	Redmond	WA	(151) 555-2272	846-78-8452 24388
Pedro Afonso	411 70th Place, Kent, WA, (170) 555-2964, 774-35-2298, 29485	Kent	WA	(170) 555-2964	774-35-2298 29485
Elizabeth Brown	971 20th Lane, Puyallup, WA, (373) 555-4134, 476-53-7164, 26417	Puyallup	WA	(373) 555-4134	476-53-7164 26417
Sven Ottlieb	676 17th Lane NE, Redmond, WA, (828) 555-1593, 548-73-8633, 27440	Redmond	WA	(828) 555-1593	548-73-8633 27440
Janine Labrune	267 95th Place SE, Seattle, WA, (949) 555-1316, 350-27-8300, 28074	Seattle	WA	(949) 555-1316	350-27-8300 28074
Ann Devon	694 53th Place, Kent, WA, (194) 555-8124, 559-74-4016, 22367	Kent	WA	(194) 555-8124	559-74-4016 22367
Roland Mendel	581 12th Street NW, Kent, WA, (103) 555-2146, 303-79-1328, 20518	Kent	WA	(103) 555-2146	303-79-1328 20518
Aria Cruz	594 85th Lane, Renton, WA, (431) 555-1376, 329-93-9992, 21498	Renton	WA	(431) 555-1376	329-93-9992 21498
Diego Roel	550 22th Lane, Renton, WA, (639) 555-6238, 918-34-5172, 25931	Renton	WA	(639) 555-6238	918-34-5172 25931
Martine Rancé	688 93th Place NW, Kent, WA, (573) 555-3571, 695-94-3479, 22424	Kent	WA	(573) 555-3571	695-94-3479 22424

FlashFill: a feature of Excel 2013 [Gulwani 2011]

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Structural Specification

- What is the space of the programs?

Synthesis Strategy

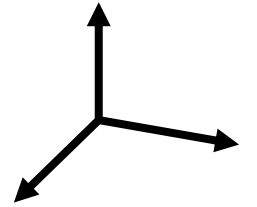
- How do we find such a program?

Github Copilot

```
// find all images  
// and add a green border around them  
// and add class "githubCopilot" to them  
function run() {
```

```
    var images = document.getElementsByTagName('img');  
    for (var i = 0; i < images.length; i++) {  
        if (images[i].className.indexOf('githubCopilot') == -1) {  
            images[i].className += ' githubCopilot';  
            images[i].style.border = '1px solid green';  
        }  
    }  
}
```

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Function signature; comment
in natural language

Structural Specification

- What is the space of the programs?

JavaScript used in Web

Synthesis Strategy

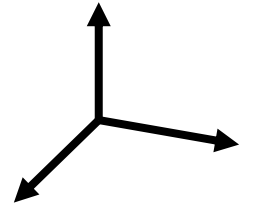
- How do we find such a program?

Large language model; next
token prediction

```
// find all images
// and add a green border around them
// and add class "githubCopilot" to them
function run() {
```

```
    var images = document.getElementsByTagName('img');
    for (var i = 0; i < images.length; i++) {
        if (images[i].className.indexOf('githubCopilot') == -1) {
            images[i].className += ' githubCopilot';
            images[i].style.border = '1px solid green';
        }
    }
}
```

Program Synthesis in Action



Behavioral Specification

- What should the program do?

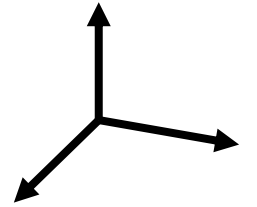
Structural Specification

- What is the space of the programs?

Synthesis Strategy

- How do we find such a program?

Program Synthesis in Action



Behavioral Specification

- What should the program do?

Description of animation,
functionalities in conversation
and markdown files

Structural Specification

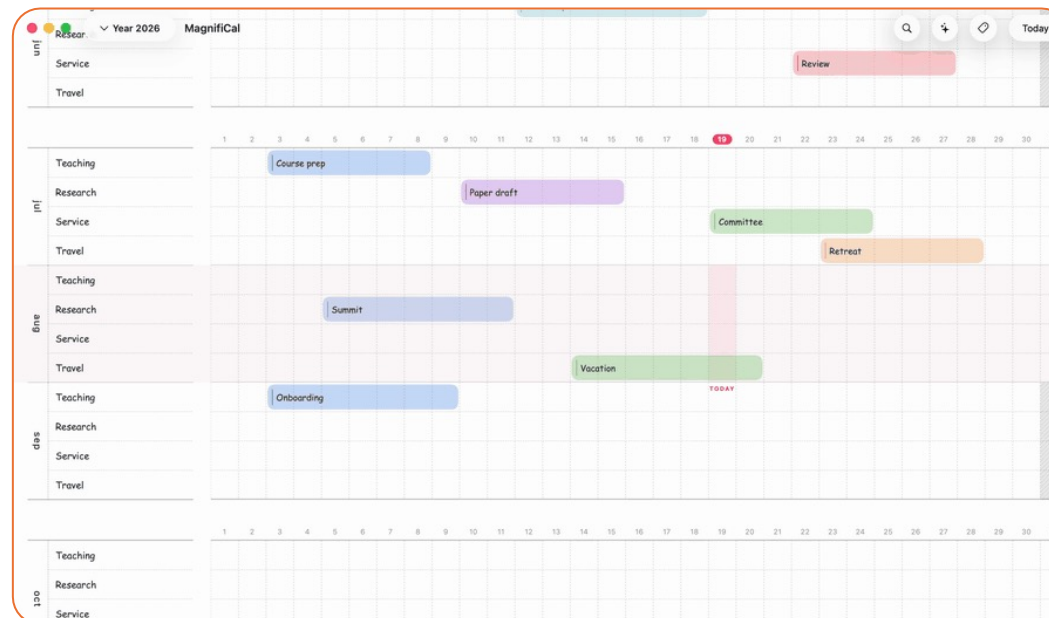
- What is the space of the programs?

A Swift program

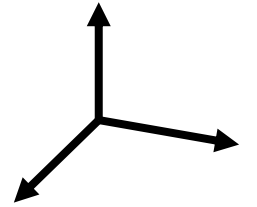
Synthesis Strategy

- How do we find such a program?

LLM-agent working with
Xcode and UI debugging
tools



Program Synthesis in Action



Behavioral Specification

- What should the program do?

Description of animation, functionalities in conversation and markdown files

Structural Specification

- What is the space of the programs?

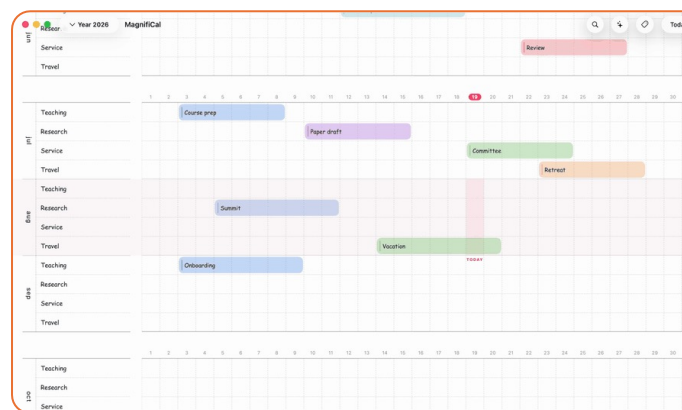
A Swift program

Synthesis Strategy

- How do we find such a program?

LLM-agent working with Xcode and UI debugging tools

```
func I prompt: I 00 index
- (a) Refined drawer behavior
  - A drawer will appear when I double click on an event. Currently, a huge drawer will be drawn out from the right
  - There is a current problem that, if I do not move the main calendar canvas, if the event is towards the right hand
    side of the screen, then the drawer will include that event; if I do shift the main calendar canvas to the left, then
    if the event is on the left hand side of the canvas, the event will go out of screen
  - The proposed solution is that, we should understand which event that we are clicking on, and the goal is to shift
    the main canvas to a point where the event is horizontally centered in the remaining space.
  - The computation is the following:
    - Say the screen width is X
    - Say the drawer's width is D,  $0 < D < X$ 
    - When the drawer is drawn out, the left-hand-side available space is of width  $X - D$ 
    - Say the event is originally horizontally located at  $0 < x < X$  (suppose it is the left-hand-side)
    - The goal would be shifting the main canvas left such that the new event location  $0' < x' < X$  is in the center of  $X - D$ ,
      meaning  $(X - D) / 2 = x'$ 
    - Therefore, the left shift delta of the main canvas is  $D - (X - D) / 2$ 
    - We do want to cap the shift delta to be above zero, because we do not want the main canvas to shift
      leftwards
    - Meaning that if an event is already placed towards the left hand side of the screen and even to the left of
      the sliding point in the first draw, we do not need to move the canvas.
  - A few notes
    - When we refine the drawer, the main canvas should be responsive and move accordingly.
    - (c) Update: We also want to upper bound the drawer
    - We do not want the shift delta to be larger than D:  $D - (X - D) / 2 > 0$ 
- (b) Scroll-pan behavior in monthly view
  - When we scroll up or down in the monthly view, we should be able to go to previous month and next month
  - The current behavior is that in the monthly view, we cannot scroll, neither left/right nor up/down
  - The scroll behavior should be the following:
    - Tap each month as if they are vertical pages
    - We are swiping across pages, like they are iPhone's homepages, just put vertical
    - When we swipe up from current month to next month, the action should be:
      - The daily timeline view first fades out (without distortion)
      - The next month's tracks come up from the bottom
    - When it gets closer to the top, the current month's track (originally stayed on the top) will move upwards
      and disappear out at the top
    - And the next month's daily view starts appearing (fade-in without distortion)
    - When we swipe down from current month to previous month, the action would be the mostly the reversed
    - The daily timeline view first fades out
    - The current month's track first moves downwards
    - While that is happening, the previous month's tracks come down from the top
    - The previous month's tracks disappear at the same place of the current month's tracks
    - The current month's tracks keep moving down and go out of the screen
    - The previous month's daily view starts appearing (fade-in)
```



```
func I prompt: I 00 index
- (a) Refined drawer behavior
  - A drawer will appear when I double click on an event. Currently, a huge drawer will be drawn out from the right
  - There is a current problem that, if I do not move the main calendar canvas, if the event is towards the right hand
    side of the screen, then the drawer will include that event; if I do shift the main calendar canvas to the left, then
    if the event is on the left hand side of the canvas, the event will go out of screen
  - The proposed solution is that, we should understand which event that we are clicking on, and the goal is to shift
    the main canvas to a point where the event is horizontally centered in the remaining space.
  - The computation is the following:
    - Say the screen width is X
    - Say the drawer's width is D,  $0 < D < X$ 
    - When the drawer is drawn out, the left-hand-side available space is of width  $X - D$ 
    - Say the event is originally horizontally located at  $0 < x < X$  (suppose it is the left-hand-side)
    - The goal would be shifting the main canvas left such that the new event location  $0' < x' < X$  is in the center of  $X - D$ ,
      meaning  $(X - D) / 2 = x'$ 
    - Therefore, the left shift delta of the main canvas is  $D - (X - D) / 2$ 
    - We do want to cap the shift delta to be above zero, because we do not want the main canvas to shift
      leftwards
    - Meaning that if an event is already placed towards the left hand side of the screen and even to the left of
      the sliding point in the first draw, we do not need to move the canvas.
  - A few notes
    - When we refine the drawer, the main canvas should be responsive and move accordingly.
    - (c) Update: We also want to upper bound the drawer
    - We do not want the shift delta to be larger than D:  $D - (X - D) / 2 > 0$ 
- (b) Scroll-pan behavior in monthly view
  - When we scroll up or down in the monthly view, we should be able to go to previous month and next month
  - The current behavior is that in the monthly view, we cannot scroll, neither left/right nor up/down
  - The scroll behavior should be the following:
    - Tap each month as if they are vertical pages
    - We are swiping across pages, like they are iPhone's homepages, just put vertical
    - When we swipe up from current month to next month, the action should be:
      - The daily timeline view first fades out (without distortion)
      - The next month's tracks come up from the bottom
    - When it gets closer to the top, the current month's track (originally stayed on the top) will move upwards
      and disappear out at the top
    - And the next month's daily view starts appearing (fade-in without distortion)
    - When we swipe down from current month to previous month, the action would be the mostly the reversed
    - The daily timeline view first fades out
    - The current month's track first moves downwards
    - While that is happening, the previous month's tracks come down from the top
    - The previous month's tracks disappear at the same place of the current month's tracks
    - The current month's tracks keep moving down and go out of the screen
    - The previous month's daily view starts appearing (fade-in)
```

Structure of the course

- **Module 1: Foundations of Program Synthesis**
 - Programming language syntax and semantics
 - Classical methods for synthesizing programs
- **Module 2: Program Synthesis in the era of Foundation Models**
 - Prompting strategies, evaluating coding LLMs
 - Iterative synthesis, MCP and tool use, agents, harness
- **Module 3: Applications of Program Synthesis**
 - Software engineering, security, theorem proving, planning, graphics
 - Robotics, HCI, AI4Science, high-performance computing
 - Other advanced topics

Week 1

- Topic:
 - Programming language **syntax & semantics**
 - **Bottom-up enumerative synthesis** from examples
- Assignment 1:
 - Will be released by Friday
 - Due Thursday of the 3rd week (Sep 17)
 - Submission on GradeScope will be opened after release